SQL SERVER PERFORMANCE SERIES · VOLUME II

Query Tuning
Real-Time Scenarios

Production scenarios in Root Cause → Fix format, with live, runnable examples on AdventureWorks2019

Praveen Madupu

Platform: Microsoft SQL Server 2019 / 2022 · Sample DB: AdventureWorks2019
Production-grade DBA reference library

Table of Contents

How to Use This Runbook 3
The universal tuning loop 3
Categories in this runbook 4
Part 1 — Indexing 5
Part 2 — SARGability & Data Types 17
Part 3 — Statistics & Cardinality Estimation 29
Part 4 — Joins 39
Part 5 — Plan Cache & Parameterization 49
Part 6 — Memory, tempdb & Spills 57
Part 7 — Concurrency & Blocking 64
Part 8 — Query Anti-Patterns 74
Part 9 — Parallelism 86
Part 10 — Schema & Storage 90
Part 11 — Advanced 96
Reference — Wait-Statistics Driven Tuning 102
Appendix — Diagnostic Toolkit 103
A. Symptom → likely cause quick index 103
B. Essential diagnostic commands 103
C. The ten habits of durable tuning 104
D. Glossary of tuning terms 104

 © PMSQLDBA

How to Use This Runbook

This is a **query-tuning runbook**: 90 real-world scenarios, each written in a strict **Symptom → Root Cause → Fix** format so you can pattern-match a production problem to a diagnosis and a remedy fast. Every example runs against the **AdventureWorks2019** sample database on SQL Server 2019 or later.

Each scenario card is laid out the same way so you can scan it under pressure:

- **Symptom** — what you actually observe (a slow query, a warning, a wait type, a blocking chain).
- **Root Cause** — the underlying reason, highlighted in red so it stands out. This is the part worth internalizing; fixes follow naturally once the cause is clear.
- **Diagnose** — the query or plan property that confirms the cause (not every card needs one).
- **Fix** — the concrete remedy, highlighted in green, with runnable T-SQL.
- **Result / Verify** — how to prove the fix worked, so you never ship a change on faith.

The universal tuning loop

Whatever the scenario, the workflow is the same. Run this loop on any slow query before reaching for a specific card:

1. **Measure the baseline.** Turn on `SET STATISTICS IO ON; SET STATISTICS TIME ON;` and capture the **actual** execution plan (`Ctrl`+`M`). Record logical reads, CPU, and elapsed time.
2. **Find the expensive step.** Highest operator cost %, thickest arrows, and any yellow/red warning icons.
3. **Check estimate vs actual rows.** Press `F4` on the suspect operator. A large gap means bad statistics — fix that first, before anything else.
4. **Classify the problem.** Scan where a seek belongs? Key/RID lookup on a fat arrow? Sort/hash spill? Implicit conversion? Blocking? Match it to a category below.
5. **Apply the matching fix** from the relevant card.
6. **Re-measure and compare.** Keep the change only if the same metrics actually improved. Weigh any new index's write cost against its read benefit.

> **BEST PRACTICE — Two rules that prevent most mistakes**
> 1) **A plan is only as good as its statistics** — fix estimates before you fix plans.
> 2) **Always measure before and after** — the execution plan tells the story, but `STATISTICS IO/TIME` (and Query Store) prove the win. Practise every change on your VMSQL01/02/03 lab before promoting it to production.

 © PMSQLDBA

Categories in this runbook

Part	Category	What it covers
1	Indexing	Missing/covering/composite indexes, lookups, residual predicates, FK indexes
2	SARGability & Data Types	Non-SARGable predicates, implicit conversions, collation, wildcards
3	Statistics & CE	Stale stats, ascending key, sniffing, variables, correlated columns
4	Joins	Loop explosions, hash spills, cartesian, type-mismatch, row explosion
5	Plan Cache	Cache bloat, RECOMPILE overuse, regressions, CE upgrades, SET options
6	Memory & tempdb	Sort/hash spills, oversized grants, tempdb allocation contention
7	Concurrency	Reader/writer blocking, lock escalation, deadlocks, NOLOCK, orphan tx
8	Anti-Patterns	Scalar UDFs, cursors, SELECT *, correlated subqueries, DISTINCT, paging
9	Parallelism	CXPACKET, cost threshold, MAXDOP, serial-forcing constructs
10	Schema & Storage	GUID clustered keys, data-type sizing, partition elimination
11	Advanced	Columnstore, rowgroup health, indexed views

© PMSQLDBA

Part 1 — Indexing

Indexes are the highest-leverage tuning tool. These scenarios cover the situations where the right index — or removing the wrong one — transforms a query.

Scenario 1 Clustered Index Scan where a Seek belongs

INDEXING

Symptom

A frequently-run lookup filtering `Sales.SalesOrderHeader` by `CustomerID` reads ~700 pages and shows a Clustered Index Scan even though it returns only a handful of rows.

ROOT CAUSE

No nonclustered index exists on the filter column. The clustered key is `SalesOrderID`, so the only way to satisfy a `CustomerID` predicate is to read the entire clustered leaf and filter each row — an O(table) operation regardless of how few rows match.

Diagnose

Capture the actual plan and `SET STATISTICS IO ON`. A Clustered Index Scan whose "Number of Rows Read" is the whole table but "Actual Number of Rows" is tiny is the classic signature of a missing seekable index.

```
SET STATISTICS IO ON;
SELECT SalesOrderID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
-- Clustered Index Scan, logical reads ~700
```

FIX

Create a nonclustered index on the filter column so the optimizer can navigate the B-Tree directly to the matching rows.

```
CREATE NONCLUSTERED INDEX IX_SOH_CustomerID
    ON Sales.SalesOrderHeader (CustomerID);

-- Re-run: Index Seek, logical reads collapse to single digits (+ a Key Lookup we
tune next)
```

Result / Verify

Logical reads drop by roughly two orders of magnitude and the operator changes to Index Seek. Confirm the seek uses a **Seek Predicate**, not a residual Predicate.

BEST PRACTICE — Design the index for the workload, not one query

© PMSQLDBA

> Before adding an index, check `sys.dm_db_index_usage_stats` and the query pattern. If many queries filter on `CustomerID` and also return the same few columns, design one covering index (next scenario) rather than several overlapping ones.

 © PMSQLDBA

Scenario 2 Key Lookup on a thick arrow INDEXING

Symptom

After adding a nonclustered index the query seeks correctly, but the plan now shows an Index Seek feeding a **Key Lookup** through a Nested Loops, and logical reads are still high under load.

ROOT CAUSE

The nonclustered index contains the filter column but not the columns the query returns. For every row the seek finds, SQL Server performs one extra B-Tree navigation into the clustered index to fetch the missing columns. On a wide result set that is one lookup per row — the cost scales with rows returned.

Diagnose

Hover the Key Lookup: its "Output List" shows exactly which columns forced the lookup. The arrow feeding it tells you how many lookups run.

FIX

Make the index **cover** the query: keep the filter column(s) as the key and add the returned columns via `INCLUDE`. Included columns live only at the leaf level, so they add no sort overhead and do not count against the key-size limit.

```
CREATE NONCLUSTERED INDEX IX_SOH_CustomerID_Covering
    ON Sales.SalesOrderHeader (CustomerID)
    INCLUDE (OrderDate, ShipDate, TotalDue, Status);

-- Plan is now a single Index Seek; the Key Lookup is gone.
```

Result / Verify

The Key Lookup and its Nested Loops disappear; the plan is a lone seek. Compare logical reads before/after — expect a large reduction.

> **TIP — Do not cover blindly**
>
> Every INCLUDEd column is duplicated in the index and maintained on every write. Cover the hot queries that run thousands of times a minute; leave a lookup in place for a report that runs twice a day.

 © PMSQLDBA

Scenario 3 Composite index with the wrong column order INDEXING

Symptom

An index on `(SalesOrderID, ProductID)` exists, yet a query filtering only by `ProductID` still scans.

ROOT CAUSE

A composite index is sorted by its **left-most column first**. With key order `(SalesOrderID, ProductID)`, the data is not physically ordered by `ProductID` alone, so a predicate on `ProductID` cannot seek — it violates the left-most prefix rule.

Diagnose

The plan shows an Index Scan (not Seek) on the composite index, or ignores it entirely in favour of the clustered index.

FIX

Order the key columns to match how queries filter: put equality-predicate columns first, then range columns. If `ProductID` is the common filter, lead with it.

```
-- Reorder to lead with the actual filter column
CREATE NONCLUSTERED INDEX IX_SOD_Product_Order
    ON Sales.SalesOrderDetail (ProductID, SalesOrderID);

SELECT * FROM Sales.SalesOrderDetail WHERE ProductID = 776;          -- Seek now
SELECT * FROM Sales.SalesOrderDetail WHERE ProductID = 776 AND SalesOrderID =
43659;  -- Seek
```

Result / Verify

The `ProductID`-only query now seeks. The `(ProductID, SalesOrderID)` prefix also serves the two-column filter for free.

> **NOTE — Equality before range**
>
> If a query does `ProductID = @p AND SalesOrderID > @s`, key the index `(ProductID, SalesOrderID)`. The equality column narrows the seek, and the range column is then satisfied within that seek. Reverse the order and the range column can no longer be seeked.

© PMSQLDBA

Scenario 4 Redundant and duplicate indexes taxing writes INDEXING

Symptom

INSERT/UPDATE/DELETE throughput on a busy table has degraded over months even though read queries look fine.

ROOT CAUSE

Multiple near-identical nonclustered indexes accumulated over time (e.g. `(CustomerID)`, `(CustomerID) INCLUDE (TotalDue)`, `(CustomerID, OrderDate)`). Each index must be maintained on every write, so redundant indexes multiply the write cost and the transaction-log volume without adding read value.

Diagnose

List indexes per table and look for overlapping leading keys; cross-check usage.

```
SELECT OBJECT_NAME(i.object_id) AS tbl, i.name, i.index_id,
       us.user_seeks, us.user_scans, us.user_lookups, us.user_updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us
       ON us.object_id = i.object_id AND us.index_id = i.index_id AND
us.database_id = DB_ID()
WHERE i.object_id = OBJECT_ID('Sales.SalesOrderHeader')
ORDER BY i.index_id;
```

FIX

Consolidate overlapping indexes into one that serves the superset, and drop the redundant ones. A single `(CustomerID) INCLUDE (OrderDate, TotalDue)` can replace three narrow variants.

```
-- Keep the widest useful covering index, drop the subsets
DROP INDEX IF EXISTS IX_SOH_Cust            ON Sales.SalesOrderHeader;
DROP INDEX IF EXISTS IX_SOH_Cust_TotalDue   ON Sales.SalesOrderHeader;
-- Retain: IX_SOH_CustomerID_Covering (CustomerID) INCLUDE (OrderDate, ShipDate,
TotalDue, Status)
```

Result / Verify

Write latency and log generation drop; read plans are unchanged because the retained index still covers them.

> **WARNING — Verify before dropping**
> `user_updates` high with `user_seeks/scans/lookups` at zero across a full business cycle is a strong drop candidate — but usage stats reset on restart. Confirm over a representative window (a month-end close counts) before removing anything, especially in SOX/PCI environments where an audit report might run quarterly.

© PMSQLDBA

Scenario 5 Missing index on a foreign key column INDEXING

Symptom

Deleting or updating a parent row is slow, and blocking spikes whenever parent rows change.

ROOT CAUSE

The child table has a foreign key but **no index on the FK column**. When a parent row is deleted or its key updated, SQL Server must scan the entire child table to check for referencing rows, and it may take broader locks while doing so.

Diagnose

Find foreign keys whose columns are not the leading key of any index.

FIX

Add a nonclustered index leading with the foreign-key column(s). This turns the referential-integrity check into a seek and reduces lock footprint.

```
-- Example: SalesOrderDetail.ProductID references Production.Product
CREATE NONCLUSTERED INDEX IX_SOD_ProductID
    ON Sales.SalesOrderDetail (ProductID);

-- Now deleting/altering a Product row seeks the child instead of scanning it.
```

Result / Verify

Parent DML latency drops and the referential-integrity check appears as a seek in the plan.

> **BEST PRACTICE — Index every FK by default**
>
> Unlike primary keys, SQL Server does **not** auto-index foreign-key columns. Indexing them is standard practice: it speeds joins to the parent, referential checks, and cascading actions.

© PMSQLDBA

Scenario 6 Filtered index the optimizer refuses to use INDEXING

Symptom

A filtered index was created for in-store orders (`WHERE OnlineOrderFlag = 0`) but the plan ignores it and scans.

ROOT CAUSE

Two common causes: (1) the query predicate is not provably a subset of the index filter (e.g. the value comes from a parameter that could be either flag), or (2) the session `SET` options at query time do not match those required for filtered indexes, so the optimizer disqualifies the index.

Diagnose

Check the query predicate matches the filter literally, and verify the required SET options.

```
-- Required ON: ANSI_NULLS, ANSI_PADDING, ANSI_WARNINGS, ARITHABORT,
--              CONCAT_NULL_YIELDS_NULL, QUOTED_IDENTIFIER
-- Required OFF: NUMERIC_ROUNDABORT
SELECT SESSIONPROPERTY('ARITHABORT'), SESSIONPROPERTY('QUOTED_IDENTIFIER');
```

FIX

Make the query predicate explicitly include the filter condition as a literal, and ensure correct SET options (SSMS defaults differ from some app drivers, which is why a query is "fast in SSMS, slow in the app").

```
SET ARITHABORT ON; SET QUOTED_IDENTIFIER ON;

SELECT CustomerID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  OnlineOrderFlag = 0                -- matches the filter literally
  AND  OrderDate >= '2013-01-01';         -- now uses the filtered index
```

Result / Verify

The plan switches to a seek on the filtered index; reads drop because the index is far smaller than the full table.

> **WARNING — The "fast in SSMS" trap**
>
> If a query is fast in SSMS but slow from the application, mismatched `ARITHABORT`/`QUOTED_IDENTIFIER` between the two is a prime suspect — it changes SET-option-dependent plan eligibility and can even produce a different cached plan.

 © PMSQLDBA

Scenario 7 Seek that still reads millions of rows (residual predicate) INDEXING

Symptom

The plan proudly shows an Index Seek, yet logical reads are enormous and the query is slow.

ROOT CAUSE

The seekable portion of the predicate is weak, so the seek positions on a huge range and the rest of the filtering happens in a **residual Predicate** applied after reading each row. "Seek" describes B-Tree navigation, not efficiency — a seek over 5 million rows that returns 12 is doing almost as much work as a scan.

Diagnose

Press F4 on the seek and compare "Number of Rows Read" to "Actual Number of Rows". A large gap means most work is residual filtering, not seeking.

```
-- Seek Predicate: what actually navigates the B-Tree (good, selective)
-- Predicate:      residual filter checked per row (bad if it removes most rows)
```

FIX

Add the residual columns into the index key (in the right order) so the filtering becomes part of the seek, not an after-the-fact scan of the seeked range.

```
-- If WHERE ProductID = 776 AND ModifiedDate >= '2014-01-01'
-- and the index is only on (ProductID), ModifiedDate is residual:
CREATE NONCLUSTERED INDEX IX_SOD_Product_Modified
    ON Sales.SalesOrderDetail (ProductID, ModifiedDate);
-- Both predicates now seek; rows read == rows returned.
```

Result / Verify

"Number of Rows Read" falls to roughly the number returned, and logical reads drop sharply.

> **NOTE — Rows Read vs Rows Returned**
>
> This single comparison is one of the fastest ways to spot a weak index. Make it a habit on every seek that is slower than expected.

 © PMSQLDBA

Scenario 8 Fragmented index slowing range scans and read-ahead INDEXING

Symptom

Range queries and reports that were fast when the index was fresh have gradually slowed; the index shows high fragmentation.

ROOT CAUSE

Repeated inserts/updates cause page splits, so the logical order of the leaf no longer matches the physical order and pages carry empty space. Fragmentation defeats sequential read-ahead and forces more page reads for the same rows.

Diagnose

Measure fragmentation and page density.

```
SELECT i.name, ips.avg_fragmentation_in_percent,
ips.avg_page_space_used_in_percent, ips.page_count
FROM sys.dm_db_index_physical_stats(DB_ID(), OBJECT_ID('Sales.SalesOrderDetail'),
NULL, NULL, 'SAMPLED') ips
JOIN sys.indexes i ON i.object_id = ips.object_id AND i.index_id = ips.index_id
WHERE ips.index_level = 0;
```

FIX

Reorganize (10–30%) or rebuild (>30%) the index, set a sensible fill factor to slow future splits, and automate the decision with a threshold-based maintenance routine.

```
ALTER INDEX IX_SOD_Product ON Sales.SalesOrderDetail REORGANIZE;
ALTER INDEX IX_SOD_Product ON Sales.SalesOrderDetail
    REBUILD WITH (ONLINE = ON, FILLFACTOR = 90, SORT_IN_TEMPDB = ON);
```

Result / Verify

Fragmentation drops, page density rises, and range-scan reads fall back to their original level.

> **BEST PRACTICE — Threshold-based, not blanket**
>
> Do not rebuild everything nightly. Use a routine (e.g. Ola Hallengren's IndexOptimize) that only acts above fragmentation and page-count thresholds, and keep statistics fresh. On your AG, remember rebuilds generate heavy log/redo traffic to secondaries.

 © PMSQLDBA

Scenario 9 ORDER BY causing a Sort because the index does not cover it

INDEXING

Symptom

A top-N query with `ORDER BY` shows a Sort operator even though an index exists on the filter column.

ROOT CAUSE

The index seeks the filter but is not ordered by the `ORDER BY` column, so SQL Server must sort the seek output. If the sort column were part of the index key in the right position, the rows would arrive pre-sorted.

FIX

Extend the index so its key ends with the ORDER BY column(s) in the required direction; the sort then disappears and TOP-N becomes a cheap ordered read.

```
-- WHERE CustomerID = @c ORDER BY OrderDate DESC
CREATE NONCLUSTERED INDEX IX_SOH_Cust_Date
    ON Sales.SalesOrderHeader (CustomerID, OrderDate DESC) INCLUDE (TotalDue);
-- Seek on CustomerID, rows already ordered by OrderDate DESC: no Sort.
```

Result / Verify

The Sort operator is gone; a TOP query returns almost instantly from the ordered index.

© PMSQLDBA

Scenario 10 Overlapping filtered indexes the optimizer cannot combine

INDEXING

Symptom

Several filtered indexes exist for different status values, yet a status query still scans or uses the wrong one.

ROOT CAUSE

Filtered indexes are only usable when the query predicate is provably a subset of the filter. A patchwork of narrow filters (`WHERE Status=1`, `=2`, …) does not help a query filtering a different or broader set, and maintaining many of them taxes writes.

FIX

Replace the patchwork with one well-designed index keyed on the status column (plus the common secondary predicate), or a single filtered index that matches the genuinely hot subset. Fewer, better-targeted indexes beat many overlapping ones.

```
-- One index serving all status values via the key:
CREATE NONCLUSTERED INDEX IX_SOH_Status_Date
    ON Sales.SalesOrderHeader (Status, OrderDate) INCLUDE (CustomerID, TotalDue);
```

Result / Verify

Status queries seek a single index; write overhead from redundant filtered indexes disappears.

© PMSQLDBA

Scenario 11 Missing index on a join column (inner side scanned) INDEXING

Symptom

A two-table join is slow; the plan seeks the outer table but **scans** the inner table for every probe.

ROOT CAUSE

The inner table has no index on the join column, so the optimizer either scans it repeatedly (nested loops) or builds a large hash. The join key must be indexed on the side being probed for an efficient seek-based join.

Diagnose

The inner input shows a Clustered Index Scan feeding the join; the join predicate is on an unindexed column.

FIX

Create a nonclustered index on the inner table's join column (INCLUDE the columns the query returns to keep it covering).

```
-- Join Sales.SalesOrderHeader h -> Sales.SalesOrderDetail d ON d.SalesOrderID =
h.SalesOrderID
-- (Detail is already clustered on SalesOrderID; for a non-clustered join key:)
CREATE NONCLUSTERED INDEX IX_SOD_SalesOrderID
    ON Sales.SalesOrderDetail (SalesOrderID) INCLUDE (ProductID, LineTotal);
```

Result / Verify

The inner scan becomes a seek; the join cost and reads drop sharply.

 © PMSQLDBA

Part 2 — SARGability & Data Types

A predicate that cannot use an index ("non-SARGable") forces a scan no matter how good your indexes are. These scenarios show the patterns that silently defeat seeks.

Scenario 12 Function wrapped around an indexed column

SARGABILITY

Symptom

A "sales in 2013" report scans the whole table despite an index on `OrderDate`.

ROOT CAUSE

Wrapping the column in a function — `WHERE YEAR(OrderDate) = 2013` — makes the predicate **non-SARGable**. SQL Server must evaluate `YEAR()` for every row before it can compare, so the sorted index is useless and a scan results.

```
SELECT COUNT(*) FROM Sales.SalesOrderHeader
WHERE  YEAR(OrderDate) = 2013;   -- function on column => scan
```

FIX

Rewrite the predicate as a half-open range against the raw column so the index can range-seek.

```
CREATE NONCLUSTERED INDEX IX_SOH_OrderDate
    ON Sales.SalesOrderHeader (OrderDate) INCLUDE (TotalDue);

SELECT COUNT(*), SUM(TotalDue)
FROM   Sales.SalesOrderHeader
WHERE  OrderDate >= '2013-01-01'
  AND  OrderDate <  '2014-01-01';   -- SARGable range => Index Seek
```

Result / Verify

Operator becomes an Index Seek; reads drop from full-table to the size of the date range.

TIP — Half-open ranges beat BETWEEN for datetimes

`>= start AND < next-period` is correct regardless of the time component; `BETWEEN start AND '2013-12-31'` silently misses rows timestamped later that day. If you must filter by an expression often, index a `PERSISTED` computed column instead.

 © PMSQLDBA

Scenario 13 Implicit conversion: NVARCHAR parameter vs VARCHAR column

SARGABILITY

Symptom

A lookup by account number scans, and the plan shows a yellow warning with `CONVERT_IMPLICIT` in the predicate.

ROOT CAUSE

The column is `VARCHAR` but the parameter is `NVARCHAR`. Because `NVARCHAR` has higher data-type precedence, SQL Server converts the **column** (every row) to `NVARCHAR`, which is non-SARGable and also degrades the cardinality estimate.

```
DECLARE @acct NVARCHAR(15) = N'10-4020-000609';
SELECT * FROM Sales.SalesOrderHeader WHERE AccountNumber = @acct;  -- scan +
warning
```

FIX

Match the parameter type to the column type (or fix the column). The conversion disappears and the seek returns.

```
DECLARE @acct VARCHAR(15) = '10-4020-000609';   -- matches column
SELECT * FROM Sales.SalesOrderHeader WHERE AccountNumber = @acct;  -- Index Seek
```

Result / Verify

The implicit-conversion warning is gone and the operator is a seek.

WARNING — ORMs cause this at scale

Entity Framework and similar frameworks default string parameters to `NVARCHAR`. Against `VARCHAR` columns this produces the same scan across an entire application. Map the ORM property to `VARCHAR` (`IsUnicode(false)` in EF) or align the column type.

© PMSQLDBA

Scenario 14 Leading-wildcard LIKE

SARGABILITY

Symptom

A name-search query using `LIKE '%son'` scans an index that would otherwise seek.

ROOT CAUSE

A wildcard at the **start** of the pattern (`%son`) means the value could begin with anything, so the sorted B-Tree cannot be positioned — the engine must examine every row. Only a trailing wildcard (`son%`) is SARGable.

FIX

If prefix search is acceptable, use a trailing wildcard. For genuine "contains" search, use a purpose-built engine: Full-Text Search, or store a reversed copy of the column and index that for suffix search.

```
-- Prefix search is SARGable
SELECT * FROM Person.Person WHERE LastName LIKE 'Ada%';   -- Seek

-- True substring search at scale: Full-Text
CREATE FULLTEXT INDEX ON Production.ProductReview (Comments)
    KEY INDEX PK_ProductReview_ProductReviewID;
SELECT * FROM Production.ProductReview
WHERE CONTAINS(Comments, '"socks"');
```

Result / Verify

Prefix queries seek; full-text queries return quickly without scanning the base table.

> **NOTE — Reversed-column trick**
>
> For suffix matching, add `RevName AS REVERSE(LastName) PERSISTED`, index it, and query `RevName LIKE REVERSE('son') + '%'`. It converts a leading-wildcard scan into a seek.

© PMSQLDBA

Scenario 15 Arithmetic performed on the indexed column SARGABILITY

Symptom

A margin filter `WHERE UnitPrice * 0.9 > 100` will not use the index on `UnitPrice`.

ROOT CAUSE

Any expression that transforms the column — arithmetic, concatenation, scalar functions — is non-SARGable because the index stores the raw column value, not the computed one.

```
WHERE UnitPrice * 0.9 > 100     -- column transformed => scan
```

FIX

Algebraically move the operation to the constant side so the column is compared bare.

```
WHERE UnitPrice > 100 / 0.9     -- column is bare => seek

-- If the expression is genuinely needed, index a PERSISTED computed column:
ALTER TABLE Sales.SalesOrderDetail
    ADD NetPrice AS (UnitPrice * (1 - UnitPriceDiscount)) PERSISTED;
CREATE INDEX IX_SOD_NetPrice ON Sales.SalesOrderDetail (NetPrice);
```

Result / Verify

The rearranged predicate seeks; the computed-column index handles the genuinely-derived case.

© PMSQLDBA

Scenario 16 OR across different columns blocking a seek

SARGABILITY

Symptom

A query with `WHERE CustomerID = @c OR SalesPersonID = @s` scans even though both columns are individually indexed.

ROOT CAUSE

A single index is sorted by one column ordering; an `OR` across two different columns cannot be satisfied by one seek because the matching rows are scattered across both indexes' orderings. The optimizer often falls back to a scan.

FIX

Split the `OR` into two seekable branches with `UNION` (or `UNION ALL` + de-dup if the sets are disjoint). Each branch seeks its own index and the results are combined.

```
SELECT SalesOrderID FROM Sales.SalesOrderHeader WHERE CustomerID = @c
UNION
SELECT SalesOrderID FROM Sales.SalesOrderHeader WHERE SalesPersonID = @s;
-- Two Index Seeks + Concatenation, instead of one scan
```

Result / Verify

The plan shows two seeks feeding a Concatenation/Merge; total reads fall well below the scan.

> **TIP — UNION vs UNION ALL**
>
> Use `UNION` when a row could match both predicates (it removes duplicates). If the two conditions are mutually exclusive, `UNION ALL` avoids the extra distinct sort.

© PMSQLDBA

Scenario 17 Mismatched collation forcing a conversion

SARGABILITY

Symptom

A join or filter between two string columns is slow and shows an implicit conversion, but both columns look like the same type.

ROOT CAUSE

The two columns have **different collations** (common after joining across databases or after a restore). SQL Server injects a `CONVERT_IMPLICIT` to reconcile collations, which is non-SARGable on the converted side.

Diagnose

Compare the collations of the two columns; a `COLLATE` clash appears in the plan's predicate.

```
SELECT name, collation_name FROM sys.columns
WHERE object_id = OBJECT_ID('dbo.SomeTable') AND name = 'Code';
```

FIX

Standardize the collations, or add an explicit `COLLATE` to the **constant/parameter** side (not the column) so the column stays seekable. The long-term fix is to align column collations.

```
-- Keep the indexed column bare; collate the other side
SELECT * FROM dbo.A JOIN dbo.B
   ON A.Code = B.Code COLLATE SQL_Latin1_General_CP1_CI_AS;
```

Result / Verify

The conversion moves off the indexed column; the seek is restored.

 © PMSQLDBA

Scenario 18 Integer column compared to a string literal SARGABILITY

Symptom

A filter like `WHERE ProductID = '776'` (string literal against an INT column) shows an implicit conversion.

ROOT CAUSE

When comparing `INT` to `VARCHAR`, integer has higher precedence, so the **string is converted to int** — which is usually fine and seekable. But the reverse (a string column compared to an int, or an int column stored as string in the app layer) converts the column and kills the seek. The danger case is a numeric-looking key stored as `VARCHAR`.

```
-- Danger: VARCHAR column, integer parameter -> column converted
DECLARE @id INT = 776;
SELECT * FROM dbo.Catalog WHERE ProductCode = @id;  -- ProductCode is VARCHAR =>
scan
```

FIX

Always pass parameters and literals in the column's own data type. Fix the application/ORM mapping rather than casting in SQL.

```
DECLARE @id VARCHAR(10) = '776';
SELECT * FROM dbo.Catalog WHERE ProductCode = @id;  -- seek
```

Result / Verify

Conversion warning clears and the operator seeks.

> **BEST PRACTICE — Type discipline at the boundary**
>
> The cleanest place to fix data-type mismatches is where parameters are declared (app/ORM/stored-proc signature). Casting inside the query only masks the problem and often keeps the column non-SARGable.

 © PMSQLDBA

Scenario 19 CASE expression in the WHERE clause blocking a seek

SARGABILITY

Symptom

A query wrapping the filter logic in a `CASE` expression scans an otherwise-indexed column.

ROOT CAUSE

A `CASE` (or any conditional expression) over the indexed column is computed per row, so the predicate is non-SARGable and cannot seek. The optimizer must evaluate the expression for the entire table.

```
WHERE CASE WHEN @mode = 1 THEN Status ELSE 0 END = @val   -- non-SARGable
```

FIX

Split the conditional logic outside the predicate — branch the query (IF/ELSE or `UNION ALL` with mutually-exclusive filters) so each branch has a plain, seekable predicate.

```
IF @mode = 1
    SELECT ... FROM Sales.SalesOrderHeader WHERE Status = @val;   -- seek
ELSE
    SELECT ... FROM Sales.SalesOrderHeader WHERE Status = 0;      -- seek
```

Result / Verify

Each branch seeks its index; the CASE-driven scan is gone.

> **TIP — Dynamic filters belong in branches**
> “Optional parameter” and “mode switch” queries are best handled by branching or `OPTION (RECOMPILE)` dynamic SQL, not by cramming the logic into a single non-SARGable predicate.

© PMSQLDBA

Scenario 20 ISNULL / COALESCE on the indexed column

SARGABILITY

Symptom

A predicate like `WHERE ISNULL(Status, 0) = 1` will not seek the index on `Status`.

ROOT CAUSE

Wrapping the column in `ISNULL`/`COALESCE` transforms it per row, making the predicate non-SARGable. The index stores raw `Status`, not the null-substituted value.

FIX

Rewrite to compare the raw column, handling NULL explicitly with an `OR ... IS NULL` (or a filtered index that excludes NULLs if that matches the need).

```
-- Instead of ISNULL(Status,0) = 1
WHERE Status = 1;                       -- seekable (NULLs excluded by = anyway)

-- If you must treat NULL as a value:
WHERE Status = 1 OR (Status IS NULL AND 0 = 1);
```

Result / Verify

The rewritten predicate seeks the index; reads drop to the matching range.

© PMSQLDBA

Scenario 21 Comparing a date column by converting it to a string SARGABILITY

Symptom

A filter like `WHERE CONVERT(varchar, OrderDate, 112) = '20130101'` scans.

ROOT CAUSE

Converting the date column to a string per row is non-SARGable and also fragile across formats. The index on `OrderDate` cannot be used because the stored value is a date, not the converted string.

FIX

Compare dates as dates using a half-open range; never convert the column to text for comparison.

```
WHERE OrderDate >= '2013-01-01' AND OrderDate < '2013-01-02';   -- seekable range
```

Result / Verify

The operator becomes a seek and results are format-independent and correct.

© PMSQLDBA

Scenario 22 Trailing-space or trimming functions defeating a seek

SARGABILITY

Symptom

A code lookup uses `WHERE RTRIM(Code) = @c` and scans.

ROOT CAUSE

`RTRIM`/`LTRIM`/`REPLACE` on the column is a per-row transformation, so the predicate is non-SARGable. (Note: for `=` comparisons, SQL Server already ignores trailing spaces under standard collations, so the trim is usually unnecessary.)

FIX

Remove the trimming function and compare the column directly; clean the data at write time if inconsistent whitespace is a real problem.

```
WHERE Code = @c;   -- seekable; trailing spaces ignored by = under default
collation
```

Result / Verify

The seek is restored; if data hygiene is the issue, normalize on insert/update instead of at query time.

© PMSQLDBA

Scenario 23 Equality OR IS NULL preventing a clean seek

SARGABILITY

Symptom

A predicate `WHERE Status = @s OR Status IS NULL` scans instead of seeking.

ROOT CAUSE

Combining an equality with an `IS NULL` on the same column produces a disjunction the optimizer often satisfies with a scan, because the two conditions target different parts of the index ordering.

FIX

Split into two seekable branches with `UNION ALL` (the branches are mutually exclusive, so no de-dup is needed), or use a filtered index if NULLs are the common case.

```
SELECT ... FROM Sales.SalesOrderHeader WHERE Status = @s
UNION ALL
SELECT ... FROM Sales.SalesOrderHeader WHERE Status IS NULL;
-- two seeks + concatenation instead of one scan
```

Result / Verify

Both branches seek; total reads fall below the original scan.

© PMSQLDBA

Part 3 — Statistics & Cardinality Estimation

The optimizer chooses plans from estimated row counts. When the estimates are wrong, everything downstream is wrong. These scenarios fix the estimates.

Scenario 24 Stale statistics producing wrong estimates

STATISTICS & CE

Symptom

A query that used to be fast has regressed; the plan looks reasonable but estimated rows are wildly off from actual rows.

ROOT CAUSE

Statistics are out of date. The optimizer sizes join types, memory grants, and parallelism from the histogram; when the data has changed but the stats have not, every downstream decision is built on a wrong guess.

Diagnose

Check when stats were last updated and how many modifications have accumulated since.

```
SELECT s.name, sp.last_updated, sp.rows, sp.rows_sampled, sp.modification_counter
FROM   sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE  s.object_id = OBJECT_ID('Sales.SalesOrderDetail')
ORDER BY sp.modification_counter DESC;
```

FIX

Update statistics (FULLSCAN for critical tables), and make sure AUTO_UPDATE_STATISTICS is on. For volatile tables, schedule proactive stats updates rather than relying on the auto threshold.

```
UPDATE STATISTICS Sales.SalesOrderDetail WITH FULLSCAN;
-- Verify database-level auto stats:
SELECT is_auto_update_stats_on, is_auto_create_stats_on
FROM sys.databases WHERE name = DB_NAME();
```

Result / Verify

Estimated vs actual rows align; the plan may switch to the correct join/grant and speed up.

> **NOTE — Estimate gap is the master signal**
>
> Almost every "the plan is wrong" problem traces back to a bad estimate. Fix the estimate first — often the optimizer then chooses the right plan on its own without any hints.

© PMSQLDBA

 © PMSQLDBA

Scenario 25 Ascending-key problem: recent rows invisible to the optimizer

STATISTICS & CE

Symptom

Queries for the most recent data (today's orders) badly underestimate rows and pick a nested-loop plan that explodes.

ROOT CAUSE

On an ever-increasing key (dates, IDENTITY), newly inserted rows sit **above** the top of the histogram until stats auto-update fires. The optimizer estimates 1 row for values beyond the histogram, so "give me today's orders" is estimated as almost nothing.

Diagnose

The estimate for a recent-date predicate is ~1 while actual is large; `modification_counter` is high relative to the auto-update threshold.

FIX

Update statistics more frequently on the hot columns, and consider Trace Flags 2371 (dynamic threshold, default on in 2016+) / behavior verified for your version. In modern SQL Server the lowered auto-update threshold usually mitigates this; targeted `UPDATE STATISTICS` on the date column is the reliable fix.

```
-- Frequent, targeted refresh of the ascending column stats
UPDATE STATISTICS Sales.SalesOrderHeader (IX_SOH_OrderDate) WITH FULLSCAN;

-- 2016+ uses the dynamic threshold automatically; confirm compat level:
SELECT compatibility_level FROM sys.databases WHERE name = DB_NAME();
```

Result / Verify

Recent-date estimates become realistic and the plan stops choosing a loop-and-seek that runs millions of times.

> **BEST PRACTICE — Hot-column stats need a schedule**
> On high-ingest tables in financial reporting, do not wait for auto-update. A nightly (or intraday) targeted stats refresh on the date/identity columns prevents the ascending-key trap during the next day's reporting.

© PMSQLDBA

Scenario 26 Parameter sniffing: fast for one value, slow for another

STATISTICS & CE

Symptom

A stored procedure runs in milliseconds for most inputs but seconds for a few; the plan is identical across calls.

ROOT CAUSE

On first execution SQL Server sniffs the parameter and caches a plan tuned for that value's selectivity. A later call with very different selectivity reuses the cached plan — a seek-and-loop plan built for a rare value is disastrous for a common one, and vice-versa.

Diagnose

Compare the compiled parameter (in the cached plan's "Parameter List") against the value that runs slowly; Query Store shows one query_id with two divergent runtime profiles.

FIX

Pick the remedy that matches the workload: recompile per call, optimize for a representative/unknown value, or force a stable good plan in Query Store.

```
-- (a) fresh plan each call (data varies a lot, proc not ultra-frequent)
... WHERE CustomerID = @CustomerID OPTION (RECOMPILE);

-- (b) one stable plan
... OPTION (OPTIMIZE FOR (@CustomerID = 29825));   -- or OPTIMIZE FOR UNKNOWN

-- (c) production-safe: force a known-good plan, no code change
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 17;
```

Result / Verify

The slow inputs now use an appropriate plan; runtime variance across parameters collapses.

> **WARNING — RECOMPILE is not free**
>
> For a proc executing thousands of times per minute, per-call compilation can cost more CPU than the bad plan wasted. Prefer Query Store plan forcing or `OPTIMIZE FOR` when frequency is high.

© PMSQLDBA

Scenario 27 Local variable defeats the histogram

STATISTICS & CE

Symptom

Replacing a literal with a local variable changes a good seek plan into a scan or a badly-sized grant.

ROOT CAUSE

The optimizer cannot sniff a **local variable** (its value is not known at compile time), so it falls back to an average **density** estimate instead of the histogram. That estimate is often far from the specific value, producing the wrong plan.

```
DECLARE @c INT = 29825;
SELECT * FROM Sales.SalesOrderHeader WHERE CustomerID = @c;  -- density estimate,
not histogram
```

FIX

Use a proper parameter (stored proc / parameterized query) so the value can be sniffed, or add `OPTION (RECOMPILE)` so the variable's runtime value is used at compile time.

```
-- Option 1: parameterize (best) — pass @c as a real parameter
-- Option 2: force recompile so the runtime value is known
DECLARE @c INT = 29825;
SELECT * FROM Sales.SalesOrderHeader WHERE CustomerID = @c
OPTION (RECOMPILE);   -- now uses the histogram for 29825
```

Result / Verify

With RECOMPILE (or true parameters) the estimate matches the value and the correct plan returns.

© PMSQLDBA

Scenario 28 Table variable estimated at one row

STATISTICS & CE

Symptom

A query joining a `@table` variable to a large table chooses a nested loop that runs millions of times.

ROOT CAUSE

Classic table variables have **no statistics** and are estimated at 1 row (pre-2019) regardless of actual population. A 1-row estimate makes the optimizer pick nested loops and tiny memory grants that are catastrophic when the variable actually holds thousands of rows.

FIX

Use a temp table (`#tmp`) which gets statistics, or on SQL Server 2019+ rely on **table-variable deferred compilation** (which estimates real cardinality), or add `OPTION (RECOMPILE)` to let the actual row count be seen at execution.

```
-- Prefer a temp table for anything non-trivial
CREATE TABLE #cust (CustomerID INT PRIMARY KEY);
INSERT #cust SELECT CustomerID FROM ...;   -- gets statistics
SELECT ... FROM #cust c JOIN Sales.SalesOrderHeader h ON h.CustomerID =
c.CustomerID;

-- Or keep @tv but force a realistic estimate:
SELECT ... OPTION (RECOMPILE);
```

Result / Verify

Estimated rows for the derived set become realistic; the join type and grant correct themselves.

> **NOTE — 2019 deferred compilation**
>
> Under database compatibility level 150+, table-variable deferred compilation estimates cardinality from actual row counts on first use — reducing (not eliminating) the historic 1-row problem. Temp tables remain the safest choice for large intermediate sets.

© PMSQLDBA

Scenario 29 Correlated columns underestimated (missing multi-column stats)

STATISTICS & CE

Symptom

A predicate on two correlated columns (e.g. City + PostalCode) underestimates rows, leading to an undersized memory grant and a spill.

ROOT CAUSE

Single-column statistics assume **independence** between predicates and multiply their selectivities. When columns are correlated (a postal code implies a city), that multiplication underestimates the true row count.

FIX

Create **multi-column statistics** (or a covering index whose leading columns are the correlated pair, which carries a multi-column stat) so the optimizer models the correlation. The 2014+ cardinality estimator also uses a milder independence assumption ("exponential backoff").

```
CREATE STATISTICS ST_Address_City_Postal
    ON Person.Address (City, PostalCode);

-- or an index that inherently provides the multi-column stat
CREATE INDEX IX_Address_City_Postal ON Person.Address (City, PostalCode);
```

Result / Verify

The combined-predicate estimate rises toward reality; grants are sized correctly and the spill disappears.

 © PMSQLDBA

Scenario 30 Default sampling too low on a very large table

STATISTICS & CE

Symptom

Estimates on a huge table are off even though statistics were “updated recently”.

ROOT CAUSE

Auto-update and a plain `UPDATE STATISTICS` use a **sampled** subset (often a small percentage on big tables). If the column is skewed, the sampled histogram misrepresents the distribution and estimates suffer.

Diagnose

Compare `rows` to `rows_sampled` in `sys.dm_db_stats_properties`; a small ratio means low sampling.

FIX

Update the critical statistics `WITH FULLSCAN` (or a higher `SAMPLE` percent) so the histogram reflects the true distribution. Balance the cost of fullscan against how critical accurate estimates are.

```
UPDATE STATISTICS Sales.SalesOrderDetail WITH FULLSCAN;
-- or a targeted higher sample:
UPDATE STATISTICS Sales.SalesOrderDetail (IX_SOD_Product) WITH SAMPLE 50 PERCENT;
```

Result / Verify

Estimates align with actuals and dependent plans (join type, grant) correct themselves.

> **NOTE — Fullscan the tables that matter**
>
> You cannot fullscan everything nightly on a large estate. Identify the tables whose estimates drive your worst plans and fullscan those on a schedule; let auto-update handle the rest.

© PMSQLDBA

Scenario 31 Synchronous auto-stats update stalling the first query

STATISTICS & CE

Symptom

The first query after a large data change is occasionally slow, then subsequent runs are fast.

ROOT CAUSE

With synchronous `AUTO_UPDATE_STATISTICS`, when a query triggers a stats refresh it **waits** for that refresh to finish before compiling — the unlucky first caller pays the cost.

FIX

Enable `AUTO_UPDATE_STATISTICS_ASYNC` so the triggering query compiles with existing stats while the refresh happens in the background (pair with proactive scheduled updates so stats never drift far).

```
ALTER DATABASE AdventureWorks2019 SET AUTO_UPDATE_STATISTICS_ASYNC ON;
```

Result / Verify

The occasional first-query stall disappears; stats still refresh, just off the critical path.

> **WARNING — Async needs proactive updates**
> Async means the triggering query uses the *old* stats that one time. Combine it with a scheduled stats job so the old stats are never badly stale — otherwise you trade a stall for a bad plan.

© PMSQLDBA

Scenario 32 Skewed subset misestimated — fix with filtered statistics

STATISTICS & CE

Symptom

Queries targeting a small, skewed subset (e.g. a rare status) get poor estimates even though full-table stats are current.

ROOT CAUSE

A single whole-table histogram averages across the whole distribution and cannot capture a rare, skewed subset accurately. Estimates for the rare value are pulled toward the average.

FIX

Create **filtered statistics** scoped to the skewed subset so the optimizer has an accurate histogram for that slice.

```
CREATE STATISTICS ST_SOH_RareStatus
    ON Sales.SalesOrderHeader (OrderDate)
    WHERE Status = 6;   -- accurate histogram for the rare slice
```

Result / Verify

Estimates for the skewed subset align with actuals; the dependent plan (grant, join) corrects itself.

> **NOTE — Filtered stats complement filtered indexes**
> Where a filtered index already exists for a subset, it carries a filtered statistic. Standalone filtered statistics are useful when you want better estimates without paying for a whole extra index.

© PMSQLDBA

Part 4 — Joins

The physical join the optimizer picks depends on estimates and available orderings. These scenarios cover the join failures you will meet most often.

Scenario 33 Nested-loop explosion from an underestimate

JOINS

Symptom

A join is slow; the plan shows Nested Loops where the inner side executes hundreds of thousands of times.

ROOT CAUSE

The optimizer estimated the outer input as tiny (bad stats or a non-sniffable variable), so it chose Nested Loops — seek the inner side once per outer row. When the outer set is actually huge, that inner seek runs an enormous number of times.

Diagnose

On the inner operator, compare "Estimated Executions" to "Actual Executions"; a big gap confirms the explosion. Trace the low estimate back to its source (stats, table variable, local variable).

FIX

Fix the estimate (update stats / temp table / RECOMPILE). Once the outer count is realistic, the optimizer will switch to Hash Match or Merge on its own. Use a join hint only as a last resort.

```
UPDATE STATISTICS dbo.OuterTable WITH FULLSCAN;
-- If a table variable feeds the outer side, switch to #temp or add OPTION
(RECOMPILE).
-- Last resort only:
SELECT ... OPTION (HASH JOIN);
```

Result / Verify

Inner executions drop to a sensible number or the plan converts to a single hash build/probe; runtime falls.

> **WARNING — Hint the estimate, not the plan**
>
> Forcing `HASH JOIN` masks the real problem and can hurt other parameter values. Correct the cardinality estimate so the optimizer keeps making good choices as data shifts.

 © PMSQLDBA

Scenario 34 Hash join spilling to tempdb

JOINS

Symptom

A large join is slow and the Hash Match operator carries a yellow spill warning; tempdb I/O spikes.

ROOT CAUSE

The hash build side needed more memory than the granted amount (grant sized from a low row estimate), so the hash table spilled to tempdb and had to be re-read — turning an in-memory operation into disk I/O.

Diagnose

The Hash Match shows a spill warning and "Actual Number of Rows" >> estimated on the build side.

FIX

Fix the estimate so the memory grant is sized correctly (update stats, multi-column stats, RECOMPILE). Reducing the build-side row count with a better filter or index also helps. Memory Grant Feedback (2017+ batch mode / 2019+ row mode) can self-correct over repeated executions.

```
UPDATE STATISTICS dbo.BigTable WITH FULLSCAN;
-- Ensure the smaller input is the build side by filtering earlier:
SELECT ... FROM small s JOIN big b ON b.k = s.k WHERE s.flag = 1;
```

Result / Verify

The spill warning clears; the join stays in memory and elapsed time drops.

> **TIP — Batch-mode Memory Grant Feedback**
>
> On modern SQL Server, repeated executions of a spilling query can auto-adjust their grant via Memory Grant Feedback. It still relies on reasonable stats to converge — do not skip the stats fix.

© PMSQLDBA

Scenario 35 Accidental Cartesian product (missing join predicate) JOINS

Symptom

A multi-table query returns far too many rows and is extremely slow; row counts look multiplicative.

ROOT CAUSE

A join condition is missing or wrong (often when adding a third table), so two tables cross-join. Every row of one multiplies against every row of the other, producing millions of rows before any filtering.

Diagnose

The plan shows a Nested Loops (Inner Join) with **no join predicate**, or a warning "no join predicate". Output cardinality is roughly the product of the inputs.

FIX

Restore the correct join predicate between the tables. Prefer explicit `JOIN ... ON` syntax over comma-joins so a missing predicate is a syntax-visible mistake.

```
-- Wrong: third table cross-joins
SELECT ... FROM A JOIN B ON A.id = B.aid JOIN C ON 1 = 1;   -- oops

-- Right: every table has a real join condition
SELECT ... FROM A
JOIN B ON B.aid = A.id
JOIN C ON C.bid = B.id;
```

Result / Verify

Row counts return to expected magnitude; the "no join predicate" warning disappears.

 © PMSQLDBA

Scenario 36 Join on mismatched data types JOINS

Symptom

A join between two tables is slow with an implicit-conversion warning on the join column.

ROOT CAUSE

The join keys are different types (e.g. `INT` in one table, `VARCHAR` in the other). SQL Server converts one side for every row, which is non-SARGable and prevents efficient seeks or merge joins.

FIX

Align the column types in the schema. As an interim measure, convert the **constant/smaller** side, never the large indexed side — but the durable fix is a schema change so the keys match.

```
-- Long-term: make the FK column the same type as the PK it references.
ALTER TABLE dbo.Child ALTER COLUMN ParentCode INT NOT NULL;  -- match parent PK
type
CREATE INDEX IX_Child_ParentCode ON dbo.Child (ParentCode);
```

Result / Verify

The conversion is gone; the join can seek/merge and the plan simplifies.

BEST PRACTICE — Model keys consistently

Foreign-key columns should always be declared with the exact type of the primary key they reference. Type drift between related tables is a recurring, avoidable source of conversion-driven scans.

 © PMSQLDBA

Scenario 37 Merge join forcing an expensive sort

JOINS

Symptom

The plan uses a Merge Join, but a large Sort operator sits in front of it and dominates the cost.

ROOT CAUSE

Merge Join requires both inputs sorted on the join key. If one input is not already ordered (no suitable index), the optimizer injects a Sort to enable the merge — and that sort can cost more than a hash join would.

FIX

Either provide an index that delivers the input pre-sorted on the join key (eliminating the sort), or let the optimizer use a Hash Match if sorting is genuinely expensive. Fixing the supporting index is usually best.

```
-- Provide sorted input so the Sort disappears and Merge stays cheap
CREATE INDEX IX_Child_JoinKey ON dbo.Child (JoinKey);
-- If sorting both sides is unavoidable and inputs are large, a hash join may be
cheaper.
```

Result / Verify

The Sort disappears (pre-sorted index) or the plan moves to a cheaper Hash Match; total cost drops.

© PMSQLDBA

Scenario 38 Row explosion from a one-to-many join before aggregation

JOINS

Symptom

A report summing header totals is inflated and slow because it joins to a detail table first.

ROOT CAUSE

Joining a header (1) to details (many) multiplies header columns across every detail row. Aggregating after the join double-counts header values and forces the engine to process a far larger intermediate set.

FIX

Pre-aggregate the "many" side to one row per key **before** joining back to the header, so the join is one-to-one and totals are correct.

```
SELECT h.SalesOrderID, h.TotalDue, d.LineCount, d.LineTotalSum
FROM   Sales.SalesOrderHeader h
JOIN (
    SELECT SalesOrderID, COUNT(*) AS LineCount, SUM(LineTotal) AS LineTotalSum
    FROM   Sales.SalesOrderDetail
    GROUP BY SalesOrderID
) d ON d.SalesOrderID = h.SalesOrderID;
```

Result / Verify

Intermediate row counts shrink, totals are correct, and the plan is markedly cheaper.

> **NOTE — Aggregate early**
> Collapsing the many-side to one row per key before the join is one of the highest-leverage rewrites for reporting queries — it reduces both the row explosion and the risk of double-counting.

© PMSQLDBA

Scenario 39 LEFT JOIN silently turned into an INNER JOIN by the WHERE clause

JOINS

Symptom

A query written as a LEFT JOIN returns fewer rows than expected (the unmatched rows vanish), and the plan looks like an inner join.

ROOT CAUSE

A predicate on the **right (outer) table** placed in the `WHERE` clause filters out the NULL-extended rows, converting the outer join back into an inner join. `WHERE r.Status = 1` eliminates every row where the right side did not match (because those have `Status = NULL`).

```
SELECT l.*, r.Status
FROM Left l
LEFT JOIN Right r ON r.lid = l.id
WHERE r.Status = 1;      -- kills the unmatched (NULL) rows => effectively INNER
```

FIX

Move the condition on the outer table into the `ON` clause, so it filters the join without removing unmatched rows; keep only left-table predicates in `WHERE`.

```
SELECT l.*, r.Status
FROM Left l
LEFT JOIN Right r ON r.lid = l.id AND r.Status = 1;   -- preserves unmatched left
rows
```

Result / Verify

The unmatched rows reappear with NULLs; row count matches the intended outer-join semantics.

> **WARNING — Correctness first, then speed**
>
> This is primarily a correctness bug that masquerades as “fewer rows.” The rule: filters on the outer table go in `ON`; filters on the preserved (left) table go in `WHERE`.

© PMSQLDBA

Scenario 40 FORCE ORDER / join hint that helped once now hurts

JOINS

Symptom

A query with `OPTION (FORCE ORDER)` or an explicit join hint used to be faster but regressed as data grew.

ROOT CAUSE

Hints freeze a decision the optimizer would otherwise adapt. `FORCE ORDER` locks the written join order; a `LOOP`/`HASH`/`MERGE` hint locks the algorithm. When data volumes or distributions shift, the frozen choice becomes wrong and the optimizer is not allowed to correct it.

FIX

Remove the hint and fix the underlying cause (usually a cardinality estimate) so the optimizer can adapt again. Keep hints only as a last resort with a documented reason and a review date.

```
-- Remove the hint, correct the estimate instead:
UPDATE STATISTICS dbo.BigTable WITH FULLSCAN;
-- Re-test without OPTION (FORCE ORDER) / (LOOP JOIN); let the optimizer choose.
```

Result / Verify

Without the hint and with good stats, the optimizer picks an order/algorithm appropriate to current data; performance recovers and stays adaptive.

© PMSQLDBA

Scenario 41 Redundant work from joining a table used only for existence

JOINS

Symptom

A query joins a large table purely to test whether related rows exist, producing duplicates and extra reads.

ROOT CAUSE

Using an inner `JOIN` for an existence check reads and carries all matching rows (and can duplicate the left side), when the query only needs to know that at least one match exists.

FIX

Replace the join with `EXISTS`, which short-circuits on the first match, avoids duplicates, and lets the optimizer use a cheap semi-join.

```
-- Instead of JOIN + DISTINCT for existence:
SELECT c.CustomerID
FROM Sales.Customer c
WHERE EXISTS (SELECT 1 FROM Sales.SalesOrderHeader h WHERE h.CustomerID =
c.CustomerID);
```

Result / Verify

Duplicates and the extra sort disappear; the plan shows a semi-join and reads fall.

© PMSQLDBA

Scenario 42 Many-to-many bridge table exploding row counts JOINS

Symptom

A query through an association/bridge table returns a multiplied row count and is slow, especially before a final aggregation.

ROOT CAUSE

Joining through a many-to-many bridge multiplies rows on both sides. Aggregating after that explosion processes a far larger set and risks double-counting measures from either parent.

FIX

Aggregate each side to the required grain **before** joining, or use `EXISTS` when you only need to know an association exists rather than to combine measures.

```
-- Pre-aggregate each side to one row per key, then join 1:1
SELECT a.Id, a.Total, b.TagCount
FROM (SELECT Id, SUM(Amount) AS Total FROM dbo.Item GROUP BY Id) a
JOIN (SELECT ItemId, COUNT(*) AS TagCount FROM dbo.ItemTag GROUP BY ItemId) b
  ON b.ItemId = a.Id;
```

Result / Verify

Intermediate row counts return to the expected magnitude; measures are no longer double-counted.

 © PMSQLDBA

Part 5 — Plan Cache & Parameterization

Compiled plans are cached and reused. These scenarios cover cache bloat, recompilation, regressions, and the sniffing/SET-option issues that flow from reuse.

Scenario 43 Plan-cache bloat from unparameterized ad-hoc SQL

PLAN CACHE

Symptom

Memory pressure and low plan reuse; thousands of single-use plans fill the cache and buffer pool has less room for data.

ROOT CAUSE

The application concatenates literal values into SQL (`WHERE CustomerID = 29825`) instead of parameterizing. Each distinct literal compiles a **new** plan that is used once and cached, wasting memory and CPU on constant recompilation.

Diagnose

Count single-use ad-hoc plans and the memory they consume.

```
SELECT objtype, COUNT(*) AS plans,
       SUM(CAST(size_in_bytes AS BIGINT))/1024/1024 AS mb,
       SUM(CASE WHEN usecounts = 1 THEN 1 ELSE 0 END) AS single_use
FROM sys.dm_exec_cached_plans
GROUP BY objtype ORDER BY mb DESC;
```

FIX

Parameterize in the application (the correct fix). As mitigations: enable "optimize for ad hoc workloads" (caches a stub on first use), and consider forced parameterization for legacy apps you cannot change.

```
-- Server-level mitigation: store a small stub until a plan is reused
EXEC sp_configure 'optimize for ad hoc workloads', 1; RECONFIGURE;

-- Database-level (use cautiously): auto-parameterize more aggressively
ALTER DATABASE AdventureWorks2019 SET PARAMETERIZATION FORCED;
```

Result / Verify

Single-use plan count and cache memory drop; plan reuse (usecounts) rises.

WARNING — Forced parameterization has side effects

© PMSQLDBA

It can reintroduce parameter-sniffing issues and hurt queries that rely on literal-specific plans (e.g. skewed data). Prefer fixing parameterization in the app; use FORCED only for unchangeable legacy workloads and test carefully.

 © PMSQLDBA

Scenario 44 OPTION (RECOMPILE) sprinkled everywhere — PLAN CACHE

Symptom

High CPU and `SQL Compilations/sec` are elevated; many procedures carry `OPTION (RECOMPILE)`.

ROOT CAUSE

RECOMPILE forces a fresh compilation on every execution. Added indiscriminately to fight one sniffing problem, it turns compilation from a rare event into a per-call tax, which on a high-frequency proc can cost more than any bad plan it avoided.

FIX

Reserve RECOMPILE for statements that genuinely need value-specific plans and run infrequently. For hot procedures, use Query Store plan forcing or `OPTIMIZE FOR` instead, or recompile only the offending **statement** rather than the whole batch.

```
-- Recompile just the sensitive statement, not the whole procedure
SELECT ... WHERE Status = @status OPTION (RECOMPILE);

-- High-frequency proc: force a stable plan in Query Store instead of recompiling.
```

Result / Verify

Compilations/sec and CPU fall; latency stabilizes.

 © PMSQLDBA

Scenario 45 A good plan regressed after a stats/index change

PLAN CACHE

Symptom

After a routine maintenance window a previously-fast query suddenly regressed to a worse plan.

ROOT CAUSE

A statistics update or schema change invalidated the cached plan; on recompile the optimizer chose a different (worse) plan — often due to a borderline cost decision or a new cardinality estimate.

Diagnose

Query Store shows the query with two plans and the exact point the runtime jumped.

```
-- Find regressed queries in Query Store
SELECT TOP 20 q.query_id, rs.avg_duration, rs.count_executions, p.plan_id
FROM sys.query_store_query q
JOIN sys.query_store_plan p ON p.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
ORDER BY rs.avg_duration DESC;
```

FIX

Force the previously-good plan in Query Store — an immediate, code-free fix — then investigate the root cause (stats sample, CE version) at leisure.

```
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 17;  -- the fast plan
```

Result / Verify

Runtime returns to baseline immediately; the forced-plan flag appears in Query Store.

BEST PRACTICE — Enable Query Store everywhere you tune

Query Store is the single best tool for catching and reverting plan regressions. Turn it on (`SET QUERY_STORE = ON`) with a sensible capture policy on every production database — it is your flight recorder.

© PMSQLDBA

Scenario 46 Cardinality-estimator upgrade regressed older queries

PLAN CACHE

Symptom

After raising database compatibility level, a set of queries regressed while others improved.

ROOT CAUSE

Compat level 120+ enables the **new cardinality estimator (CE)**, which changed independence/correlation assumptions. Most queries benefit, but some plans built on old-CE assumptions regress under the new model.

FIX

Keep the new CE globally but revert the specific regressed queries to legacy CE via a query hint or plan guide — not the whole database. `Database Scoped Configuration` and Query Store give you surgical control.

```
-- Per-query legacy CE without changing the whole DB
SELECT ... OPTION (USE HINT ('FORCE_LEGACY_CARDINALITY_ESTIMATION'));

-- Or test both CEs quickly:
SELECT ... OPTION (USE HINT ('FORCE_DEFAULT_CARDINALITY_ESTIMATION'));
```

Result / Verify

Regressed queries recover under legacy CE while the rest of the workload keeps the new-CE gains.

> **TIP — Use Query Store to compare CEs**
> Before upgrading compat level in production, run the workload under the new CE with Query Store on. You get a per-query before/after comparison and a ready-made list of regressions to hint or force.

© PMSQLDBA

Scenario 47 Wrong plan cached because of SET-option differences

PLAN CACHE

Symptom

The same query text is fast from one client and slow from another; the cache holds two plans for it.

ROOT CAUSE

Plans are cached per **SET-options** context. A client with `ARITHABORT OFF` (some drivers) gets a different cache entry — and possibly a different, worse plan — than SSMS which defaults `ARITHABORT ON`.

Diagnose

Look for duplicate plans for the same text with different `set_options` in `sys.dm_exec_plan_attributes`.

FIX

Standardize session settings across clients (set `ARITHABORT ON`, `QUOTED_IDENTIFIER ON` in the connection or app). This also removes the "fast in SSMS, slow in app" illusion.

```
-- Ensure the app connection matches SSMS defaults
SET ARITHABORT ON;
SET QUOTED_IDENTIFIER ON;
SET ANSI_NULLS ON;
```

Result / Verify

Both clients resolve to the same, correct plan; the duplicate cache entry disappears.

© PMSQLDBA

Scenario 48 EXEC with string concatenation defeating plan reuse

PLAN CACHE

Symptom

A procedure that builds dynamic SQL with `EXEC(@sql)` shows many single-use plans and high compilation.

ROOT CAUSE

Concatenating parameter values directly into a dynamic string produces a unique statement per value, so every execution compiles a new plan — no reuse, plus SQL-injection risk.

```
SET @sql = N'SELECT * FROM Sales.SalesOrderHeader WHERE CustomerID = ' + CAST(@c
AS nvarchar);
EXEC (@sql);   -- unique text each call => new plan each time
```

FIX

Use `sp_executesql` with **parameters** so the statement text is stable and its plan is reused (and the query is injection-safe).

```
EXEC sys.sp_executesql
    N'SELECT * FROM Sales.SalesOrderHeader WHERE CustomerID = @c',
    N'@c INT', @c = @c;   -- stable text, one reusable plan
```

Result / Verify

The statement compiles once and reuses; single-use plan count and compilations/sec fall.

> **BEST PRACTICE — sp_executesql over EXEC**
>
> Parameterized `sp_executesql` gives plan reuse and blocks SQL injection. Reserve string-concatenated `EXEC` for genuinely dynamic structure (table/column names), and even then whitelist identifiers.

© PMSQLDBA

Scenario 49 Temp table DDL causing repeated recompilations inside a procedure

PLAN CACHE

Symptom

A stored procedure recompiles frequently; `SP:Recompile` events fire and CPU is elevated.

ROOT CAUSE

Interleaving DDL and DML on a temp table inside the proc (create, then insert, then reference, then alter) triggers statement-level recompiles as schema/stats change. Creating indexes on the temp table after populating it, or mixing DDL with DML, is a common trigger.

FIX

Define the temp table and its indexes **up front**, populate it in one pass, and avoid altering it later. Where appropriate, use a table variable for tiny sets (no recompiles) or `KEEPFIXED PLAN` when the recompiles are stats-driven and unnecessary.

```
-- Create structure + indexes first, then load once:
CREATE TABLE #t (id INT PRIMARY KEY, amt MONEY);
INSERT #t (id, amt) SELECT ...;      -- single population
-- reference #t afterward; do not ALTER it mid-proc
```

Result / Verify

Recompile events drop; the procedure's plan stabilizes and CPU falls.

© PMSQLDBA

Part 6 — Memory, tempdb & Spills

Sorts and hashes need memory grants sized from estimates. Get the estimate wrong and work spills to tempdb. These scenarios keep operations in memory.

Scenario 50 Sort operator spilling to tempdb

MEMORY & TEMPDB

Symptom

A sorted/paged grid is slow; the plan shows a Sort with a yellow spill warning and tempdb usage climbs under load.

ROOT CAUSE

The Sort requested a memory grant sized from the row estimate. When the estimate is too low, the sort cannot fit in memory and **spills** intermediate data to tempdb, adding physical I/O to a formerly in-memory operation.

Diagnose

Hover the Sort: a spill warning plus estimated-rows << actual-rows confirms it.

FIX

Remove the sort entirely by providing an index already in the required order, and update statistics so any remaining grants are correct.

```
CREATE NONCLUSTERED INDEX IX_SOH_TotalDue_Desc
    ON Sales.SalesOrderHeader (TotalDue DESC) INCLUDE (CustomerID);
UPDATE STATISTICS Sales.SalesOrderHeader WITH FULLSCAN;

SELECT SalesOrderID, CustomerID, TotalDue
FROM   Sales.SalesOrderHeader
ORDER BY TotalDue DESC;   -- reads the index in order: no Sort, no spill
```

Result / Verify

The Sort disappears from the plan; tempdb writes and elapsed time drop.

> **NOTE — Sort is blocking and memory-hungry**
>
> A Sort must consume its entire input before emitting the first row, and it holds its grant for the duration. Eliminating it with an ordered index helps both latency and concurrency.

© PMSQLDBA

Scenario 51 Excessive memory grant starving concurrency

MEMORY & TEMPDB

Symptom

A query requests a very large memory grant; under concurrency, other queries queue on `RESOURCE_SEMAPHORE` waits.

ROOT CAUSE

An **over-estimate** (or a very wide sort/hash) caused the optimizer to request far more memory than the query actually needs. Large grants reduce the memory available to concurrent queries, which then wait for a grant.

Diagnose

Compare granted vs used memory; look for `RESOURCE_SEMAPHORE` waits.

```
SELECT session_id, requested_memory_kb, granted_memory_kb, used_memory_kb,
query_cost
FROM sys.dm_exec_query_memory_grants ORDER BY granted_memory_kb DESC;
```

FIX

Fix the over-estimate (stats), narrow the columns being sorted/hashed (avoid `SELECT *`), and rely on Memory Grant Feedback to right-size repeated executions. Resource Governor can cap grants for a workload group as a guardrail.

```
-- Narrow the row width so the grant shrinks
SELECT SalesOrderID, TotalDue FROM ... ORDER BY TotalDue;  -- not SELECT *
UPDATE STATISTICS ... WITH FULLSCAN;
-- Optional guardrail:
-- Resource Governor workload group with REQUEST_MAX_MEMORY_GRANT_PERCENT
```

Result / Verify

Granted memory approaches used memory; `RESOURCE_SEMAPHORE` waits subside and concurrency improves.

> **WARNING — SELECT * inflates grants**
>
> Every column you carry through a Sort or Hash widens the row and enlarges the memory grant. Returning only needed columns is both a read and a memory optimization.

© PMSQLDBA

Scenario 52 tempdb allocation contention (PFS/GAM/SGAM latching)

MEMORY & TEMPDB

Symptom

Under heavy concurrency, sessions wait on `PAGELATCH_UP`/`PAGELATCH_EX` on tempdb allocation pages (2:1:1, 2:1:3, etc.).

ROOT CAUSE

Many sessions creating temp objects simultaneously contend on the shared allocation pages (PFS/GAM/SGAM) of tempdb. With too few data files, all sessions hammer the same allocation bitmaps.

Diagnose

Waits on `PAGELATCH_*` against database_id 2 on low page numbers indicate allocation contention.

FIX

Give tempdb multiple equally-sized data files (a common starting point is one file per logical core up to 8, then adjust), enable uniform autogrowth, and on 2016+ benefit from the built-in allocation improvements. Reduce needless temp-object churn in code.

```
-- Add/size tempdb files evenly (illustrative)
ALTER DATABASE tempdb MODIFY FILE (NAME='tempdev', SIZE=1024MB, FILEGROWTH=256MB);
ALTER DATABASE tempdb ADD FILE (NAME='tempdev2', FILENAME='...\tempdev2.ndf',
SIZE=1024MB, FILEGROWTH=256MB);
-- repeat for the target number of equally-sized files
```

Result / Verify

Allocation-page latch waits fall and throughput on temp-heavy workloads rises.

BEST PRACTICE — Even files, even growth

tempdb files must be equal in size with equal autogrowth so the proportional-fill algorithm spreads allocations evenly. One giant file plus small ones defeats the purpose.

© PMSQLDBA

Scenario 53 Hash aggregate spilling on a large GROUP BY

MEMORY & TEMPDB

Symptom

A `GROUP BY` over many rows is slow with a Hash Match (Aggregate) spill warning.

ROOT CAUSE

The hash table built for grouping exceeded its memory grant (usually an under-estimate of distinct groups) and spilled to tempdb.

FIX

Fix the estimate, reduce the grouped row width, and where the input can be delivered pre-sorted on the grouping key, a **Stream Aggregate** (needs sorted input, tiny memory) avoids the hash entirely.

```
-- Provide an index ordered by the grouping key -> Stream Aggregate, no hash spill
CREATE INDEX IX_SOD_Product ON Sales.SalesOrderDetail (ProductID) INCLUDE
(LineTotal);

SELECT ProductID, SUM(LineTotal)
FROM   Sales.SalesOrderDetail
GROUP BY ProductID;   -- can use Stream Aggregate over the ordered index
```

Result / Verify

The hash spill clears (or the plan switches to Stream Aggregate); elapsed time and tempdb writes drop.

© PMSQLDBA

Scenario 54 TOP row-goal producing a bad plan

MEMORY & TEMPDB

Symptom

Adding `TOP (n)` or `SET ROWCOUNT` unexpectedly changes a good plan into a slow one that never seems to "find" the rows.

ROOT CAUSE

A `TOP` sets a **row goal**: the optimizer assumes it can stop early and picks a plan (often a nested-loop seek) optimized to return the first N rows quickly. If the rows that satisfy the filter are sparse or at the "end," the engine reads far more than expected chasing the goal.

Diagnose

Compare estimated vs actual on the seek feeding the TOP; a huge actual with a tiny estimate signals a mis-served row goal.

FIX

Provide an index that supports both the filter and the required order so the row goal is met by a genuine ordered seek; or, when the row goal misleads, hint it off.

```
-- Best: index that satisfies filter + ORDER BY so TOP reads exactly N rows
CREATE INDEX IX_SOH_Status_Date ON Sales.SalesOrderHeader (Status, OrderDate)
INCLUDE (TotalDue);

-- If the row goal itself is the problem:
SELECT TOP (10) ... ORDER BY OrderDate
OPTION (USE HINT ('DISABLE_OPTIMIZER_ROWGOAL'));
```

Result / Verify

The TOP query reads roughly N rows via the ordered index instead of over-reading; runtime becomes predictable.

© PMSQLDBA

Scenario 55 Window function forcing a worktable spool

MEMORY & TEMPDB

Symptom

A query using `ROW_NUMBER()`/`SUM() OVER(...)` is slow and shows a Table Spool plus a Sort with high tempdb use.

ROOT CAUSE

Window functions require the input ordered by the `PARTITION BY`/`ORDER BY` columns. Without a supporting index the optimizer adds a Sort and often a worktable Spool to compute the window, and an under-estimate makes the sort spill.

FIX

Create an index whose key matches the window's `PARTITION BY` then `ORDER BY` columns so the rows arrive pre-ordered, removing the Sort (and often the spool).

```
-- For: ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC)
CREATE INDEX IX_SOH_Cust_Date
    ON Sales.SalesOrderHeader (CustomerID, OrderDate DESC) INCLUDE (TotalDue);
-- Window now reads pre-sorted input: no Sort, no worktable spill.
```

Result / Verify

The Sort/Spool disappears and tempdb writes drop; the windowed query speeds up markedly.

> **TIP — Match the index to the window**
>
> The winning index key is exactly (PARTITION BY columns, then ORDER BY columns, in the ORDER BY direction). Add the measures as INCLUDE columns to keep it covering.

 © PMSQLDBA

Scenario 56 Memory Grant Feedback reset by OPTION (RECOMPILE) MEMORY & TEMPDB

Symptom

A query still spills every run despite Memory Grant Feedback being available on your SQL Server version.

ROOT CAUSE

Memory Grant Feedback learns the right grant across **repeated reuse of a cached plan**. If every execution recompiles (`OPTION (RECOMPILE)` or constant plan-cache eviction), feedback never accumulates, so the grant is never corrected and the spill persists.

FIX

Let the plan be cached and reused so feedback can converge; remove unnecessary RECOMPILE, and fix the underlying estimate (stats) so the initial grant is closer. Reserve RECOMPILE for genuinely value-sensitive statements.

```
-- Remove RECOMPILE where it is not needed so the cached plan can learn its grant.
-- Fix the estimate so the first grant is already reasonable:
UPDATE STATISTICS dbo.BigTable WITH FULLSCAN;
```

Result / Verify

Across repeated executions the granted memory converges toward the used memory and the spill stops recurring.

© PMSQLDBA

Part 7 — Concurrency & Blocking

Correct results under concurrency without crippling blocking is a balance of isolation level, transaction scope, and lock granularity. These scenarios strike it.

Scenario 57 Reader blocking writer under default isolation

CONCURRENCY

Symptom

Reports and OLTP writes block each other; sessions wait on `LCK_M_S` / `LCK_M_X` and users report intermittent hangs.

ROOT CAUSE

Under the default `READ COMMITTED` (pessimistic) isolation, readers take shared locks that block writers and vice-versa. A long-running report holds shared locks that stall updates on the same rows/pages.

Diagnose

Find the blocking chain and the lock types involved.

```
SELECT r.session_id, r.blocking_session_id, r.wait_type, r.wait_time,
       t.text AS running_sql
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.blocking_session_id <> 0;
```

FIX

Enable **Read Committed Snapshot Isolation (RCSI)** so readers use row-versioning and no longer block writers (and vice-versa). Test tempdb capacity first, since versions live in tempdb.

```
-- Requires a brief exclusive moment on the DB
ALTER DATABASE AdventureWorks2019 SET READ_COMMITTED_SNAPSHOT ON;
-- Readers now see the last committed version instead of taking blocking shared
locks.
```

Result / Verify

Reader/writer blocking largely disappears; `LCK_M_S` waits fall. Monitor tempdb and the version store.

> **WARNING — RCSI shifts cost to tempdb**
>
> Row versioning stores previous versions in tempdb. Before enabling RCSI in production, ensure tempdb is well-sized and on fast storage, and watch the version-store cleanup. It is a database-wide change — validate on your VMSQL lab first.

© PMSQLDBA

 © PMSQLDBA

Scenario 58 Lock escalation to a table lock

CONCURRENCY

Symptom

A large UPDATE/DELETE suddenly blocks the whole table; other sessions cannot touch any row.

ROOT CAUSE

When a statement acquires more than ~5,000 locks on an object, SQL Server **escalates** row/page locks to a single table lock to save memory. That table lock blocks all other access until the transaction ends.

Diagnose

Blocking on the whole object with a `OBJECT`-level `X`/`IX` lock; the offending statement modifies many rows in one transaction.

FIX

Modify in **smaller batches** so the lock count stays under the escalation threshold, keeping locks at row/page granularity and letting other sessions interleave.

```
-- Batch a big delete to avoid escalation and long blocking
WHILE 1 = 1
BEGIN
    DELETE TOP (2000) FROM dbo.BigLog WHERE CreatedDate < '2023-01-01';
    IF @@ROWCOUNT = 0 BREAK;
END
```

Result / Verify

The table-level lock no longer appears; concurrent access continues while the batches run.

> **TIP — Batching also tames the log**
> Small committed batches keep the transaction log from ballooning and, on your AG, reduce redo pressure on the secondaries — a double win for large maintenance operations.

© PMSQLDBA

Scenario 59 Deadlock from inconsistent object access order

CONCURRENCY

Symptom

Intermittent "transaction was deadlocked" (error 1205) errors under concurrency.

ROOT CAUSE

Two transactions lock the same two resources in **opposite order**: T1 locks A then wants B; T2 locks B then wants A. Each waits on the other, and the deadlock monitor kills one as the victim.

Diagnose

Capture the deadlock graph (system_health XEvent session already records it).

```
-- Read recent deadlock graphs from system_health
SELECT XEvent.value('(data/value)[1]','nvarchar(max)') AS deadlock_xml
FROM (SELECT CAST(target_data AS XML) td
      FROM sys.dm_xe_session_targets st
      JOIN sys.dm_xe_sessions s ON s.address = st.event_session_address
      WHERE s.name = 'system_health') AS d
CROSS APPLY td.nodes('//event[@name="xml_deadlock_report"]') AS X(XEvent);
```

FIX

Make all code paths acquire objects in a **consistent order**, keep transactions short, and add a covering index so the conflicting statements finish faster (less time holding locks). Retry logic handles the rare residual deadlock.

```
-- 1) Always touch tables in the same order (e.g. Parent then Child) everywhere.
-- 2) Keep transactions short; do not hold locks across app round-trips.
-- 3) Index to shrink lock duration.
-- 4) Add application retry on error 1205.
```

Result / Verify

Deadlock frequency drops to near zero; the deadlock graph shows the ordering conflict is gone.

© PMSQLDBA

Scenario 60 NOLOCK causing dirty and duplicated rows

CONCURRENCY

Symptom

A report intermittently shows wrong totals, missing rows, or duplicate rows, and developers "fixed" earlier blocking with `WITH (NOLOCK)`.

ROOT CAUSE

`NOLOCK` (READ UNCOMMITTED) reads uncommitted data and can read rows twice or skip rows entirely when pages move during a scan (allocation-order reads). It trades correctness for reduced blocking — dangerous for financial reporting.

FIX

Remove `NOLOCK` from correctness-sensitive queries. Solve the underlying blocking properly with **RCSI/snapshot isolation**, which gives non-blocking reads *without* dirty-read anomalies.

```
-- Replace NOLOCK hints with a consistent, non-blocking isolation:
ALTER DATABASE AdventureWorks2019 SET READ_COMMITTED_SNAPSHOT ON;
-- Then drop WITH (NOLOCK) from queries; reads are versioned and correct.
```

Result / Verify

Report totals become consistent and reproducible; blocking stays low thanks to versioning.

WARNING — NOLOCK is not "go faster"

In SOX/PCI-audited reporting, `NOLOCK` can silently produce incorrect figures. Treat it as a correctness hazard, not a performance setting. RCSI is the right tool for non-blocking reads.

© PMSQLDBA

Scenario 61 Orphaned open transaction blocking everyone

CONCURRENCY

Symptom

A single old session (`sleeping`, `awaiting command`) holds locks and blocks a growing chain of sessions; the log will not truncate.

ROOT CAUSE

An application opened a transaction and never committed/rolled back (a bug, a lost connection with pooling, or `SET IMPLICIT_TRANSACTIONS ON`). Its locks persist and block others; the active transaction also pins the log.

Diagnose

Find the oldest open transaction and what it holds.

```
SELECT s.session_id, s.status, s.last_request_end_time,
       DB_NAME(dt.database_id) AS db, dt.database_transaction_begin_time
FROM sys.dm_tran_session_transactions st
JOIN sys.dm_tran_database_transactions dt ON dt.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions s ON s.session_id = st.session_id
ORDER BY dt.database_transaction_begin_time;
```

FIX

Kill the orphaned session to release locks, then fix the application: ensure every transaction commits/rolls back, disable stray `IMPLICIT_TRANSACTIONS`, and set connection-pool reset. Add a monitor/alert for long-open transactions.

```
-- Immediate relief
KILL 57;   -- the orphaned SPID

-- Prevent recurrence in app connections
SET IMPLICIT_TRANSACTIONS OFF;   -- unless intentionally used with matching
COMMITs
```

Result / Verify

The blocking chain clears instantly; the log can truncate again. Long-transaction alerting catches recurrences.

© PMSQLDBA

Scenario 62 Last-page insert contention (hotspot) on a sequential key

CONCURRENCY

Symptom

A high-ingest table clustered on an ever-increasing key shows `PAGELATCH_EX` waits on the final page under many concurrent inserts.

ROOT CAUSE

Every insert targets the **same last page** of a sequential clustered index, so concurrent inserters queue on the exclusive page latch. Ironically this is the opposite failure mode of a random GUID key — perfect ordering creates a single insert hotspot.

Diagnose

High `PAGELATCH_EX` waits concentrated on one object's trailing page under concurrent insert load.

FIX

Spread inserts across multiple pages. Options: use `OPTIMIZE_FOR_SEQUENTIAL_KEY` (2019+), hash-partition the insert point, or (carefully) introduce a leading distribution column. Measure before choosing — the 2019 option is the least invasive.

```
-- SQL Server 2019+: built-in mitigation for last-page contention
ALTER TABLE dbo.HotLog
    SET (OPTIMIZE_FOR_SEQUENTIAL_KEY = ON);
```

Result / Verify

`PAGELATCH_EX` waits on the trailing page fall and insert throughput scales with concurrency.

> **NOTE — Two opposite hotspots**
> Random GUID keys fragment and split (Part 10); perfectly sequential keys create a last-page latch hotspot. `OPTIMIZE_FOR_SEQUENTIAL_KEY` targets the latter without giving up the benefits of an ordered key.

© PMSQLDBA

Scenario 63 Deadlock caused by a missing foreign-key index

CONCURRENCY

Symptom

Deadlocks occur between a parent-row modification and child-table activity; the graph shows range/table locks on the child.

ROOT CAUSE

Without an index on the child's foreign-key column, checking or locking child rows for a parent change scans and takes broad locks, which collide with concurrent child DML and deadlock.

FIX

Index the foreign-key column so the referential check becomes a narrow seek with minimal locking, removing the collision surface.

```
CREATE NONCLUSTERED INDEX IX_SOD_ProductID
    ON Sales.SalesOrderDetail (ProductID);   -- FK to Production.Product
```

Result / Verify

The child access seeks instead of scanning; the deadlock pattern disappears.

> **BEST PRACTICE — FK indexes prevent deadlocks too**
> Beyond speeding joins and deletes, indexing foreign keys shrinks the lock footprint of referential checks — a frequent, avoidable source of deadlocks in write-heavy schemas.

© PMSQLDBA

Scenario 64 Update conflict (error 3960) under snapshot isolation

CONCURRENCY

Symptom

Under `SNAPSHOT` isolation, updates intermittently fail with error 3960: “snapshot isolation transaction aborted due to update conflict.”

ROOT CAUSE

Snapshot isolation is optimistic: a transaction reads a consistent version, but if another transaction commits a change to the same row before it writes, the writer is aborted to preserve consistency. This is expected behavior, not corruption.

FIX

Add application **retry logic** for 3960 (re-run the transaction), keep write transactions short to shrink the conflict window, and consider `READ COMMITTED SNAPSHOT` instead where you need non-blocking reads without the write-conflict semantics.

```
-- Retry pattern (pseudo-T-SQL)
-- BEGIN TRY  BEGIN TRAN ... COMMIT  END TRY
-- BEGIN CATCH IF ERROR_NUMBER() = 3960 -> wait briefly and retry END CATCH

-- If you only need non-blocking reads, prefer RCSI over full SNAPSHOT:
ALTER DATABASE AdventureWorks2019 SET READ_COMMITTED_SNAPSHOT ON;
```

Result / Verify

Transient 3960s are retried transparently; user-visible failures stop. Choosing RCSI avoids write conflicts where full snapshot semantics are not required.

© PMSQLDBA

Scenario 65 Index maintenance blocking the workload (offline rebuild)

CONCURRENCY

Symptom

During the maintenance window an index REBUILD blocks all access to the table; users report timeouts.

ROOT CAUSE

A default (offline) `REBUILD` takes a schema-modification lock for the duration, blocking reads and writes on the object. On a large index this can block the workload for a long time.

FIX

Use `ONLINE = ON` (Enterprise/Developer) so the rebuild allows concurrent access, and/or `REORGANIZE` (always online) for moderate fragmentation. On 2019+ consider `RESUMABLE = ON` to pause/resume long rebuilds.

```
ALTER INDEX IX_SOD_Product ON Sales.SalesOrderDetail
    REBUILD WITH (ONLINE = ON, RESUMABLE = ON, MAX_DURATION = 30 MINUTES);
-- REORGANIZE is always online and lighter for 10-30% fragmentation:
ALTER INDEX IX_SOD_Product ON Sales.SalesOrderDetail REORGANIZE;
```

Result / Verify

The workload continues during maintenance; the schema-modification lock no longer blocks the table for the full rebuild.

WARNING — ONLINE has trade-offs

Online rebuilds use extra space and tempdb, take a brief Sch-M lock at start/finish, and generate more log/redo to AG secondaries. Schedule accordingly and validate on your VMSQL01/02/03 lab.

© PMSQLDBA

Part 8 — Query Anti-Patterns

Some queries are slow by construction. These scenarios rewrite the common anti-patterns into set-based, index-friendly form.

Scenario 66 Scalar UDF invoked per row

ANTI-PATTERNS

Symptom

A query with a scalar user-defined function in the SELECT/WHERE runs far slower than the raw logic, and the plan hides the cost.

ROOT CAUSE

Pre-2019, a scalar UDF executes **once per row**, cannot be costed properly by the optimizer, and forces the whole query into a serial (non-parallel) plan. The per-row invocation dominates on large sets.

```
-- dbo.fnTaxAmount(...) called for every row
SELECT SalesOrderID, dbo.fnTaxAmount(SubTotal, TaxRate) AS Tax
FROM   Sales.SalesOrderHeader;   -- serial plan, per-row calls
```

FIX

Inline the logic as an expression, convert the scalar UDF to an **inline table-valued function** used with `CROSS APPLY`, or on SQL Server 2019+ rely on **scalar UDF inlining** (compat level 150) which folds many UDFs into the query.

```
-- Inline the expression directly
SELECT SalesOrderID, SubTotal * TaxRate AS Tax
FROM   Sales.SalesOrderHeader;

-- Or an inline TVF with APPLY
SELECT h.SalesOrderID, t.Tax
FROM   Sales.SalesOrderHeader h
CROSS APPLY dbo.fnTaxAmount_Inline(h.SubTotal, h.TaxRate) t;
```

Result / Verify

The plan can go parallel again and per-row call overhead vanishes; elapsed time drops sharply.

> **TIP — 2019 scalar UDF inlining**
>
> Under compat level 150, many T-SQL scalar UDFs are automatically inlined, removing the per-row penalty with no rewrite. Some UDFs are ineligible (e.g. those using certain constructs) — verify with the actual plan.

© PMSQLDBA

 © PMSQLDBA

Scenario 67 Row-by-row cursor instead of set-based logic

ANTI-PATTERNS

Symptom

A nightly job using a cursor takes hours; CPU is low but the job crawls.

ROOT CAUSE

A cursor processes one row per iteration with per-row overhead (fetch, context switch, individual statements). SQL Server is optimized for **set-based** operations; procedural row-by-row processing forfeits that.

FIX

Rewrite the cursor loop as a single set-based statement (`UPDATE ... FROM`, `INSERT ... SELECT`, `MERGE`). For genuinely huge operations, batch the set-based statement rather than iterating rows.

```
-- Instead of a cursor updating each row:
UPDATE d
SET    d.LineTotal = d.UnitPrice * d.OrderQty * (1 - d.UnitPriceDiscount)
FROM   Sales.SalesOrderDetail d;

-- For very large tables, batch the set-based update (see lock-escalation
scenario).
```

Result / Verify

A multi-hour cursor typically collapses to seconds/minutes as one set operation.

BEST PRACTICE — Set-based first

Reach for a cursor only when the logic is inherently sequential (e.g. running a stored proc per row with side effects). Almost all data transformations are expressible set-based.

© PMSQLDBA

Scenario 68 SELECT * dragging unnecessary and LOB columns

ANTI-PATTERNS

Symptom

A grid query is slower than expected and prevents a covering index from being used.

ROOT CAUSE

`SELECT *` returns every column, including wide/LOB columns the app never displays. It inflates I/O and memory grants, and it defeats covering indexes because the index would have to include *all* columns to avoid a lookup.

FIX

Select only the columns actually needed. A narrow column list lets a covering index satisfy the query and shrinks sorts/grants.

```
-- Instead of SELECT *
SELECT SalesOrderID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  CustomerID = 29825;
-- Now a (CustomerID) INCLUDE (OrderDate, TotalDue) index fully covers it.
```

Result / Verify

Reads and memory grant drop; the covering index eliminates the lookup.

© PMSQLDBA

Scenario 69 Correlated subquery evaluated once per outer row

ANTI-PATTERNS

Symptom

A query with a correlated subquery in the SELECT list is slow and shows a repeated inner seek.

ROOT CAUSE

The correlated subquery references the outer row, so it is (logically) evaluated for **every** outer row. On a large outer set this becomes a per-row lookup, similar to a scalar UDF.

FIX

Rewrite as a `JOIN` or `APPLY` with a single pre-aggregated derived table so the inner work happens once, set-based.

```
-- Slow: correlated subquery per row
SELECT h.SalesOrderID,
       (SELECT SUM(d.LineTotal) FROM Sales.SalesOrderDetail d
        WHERE d.SalesOrderID = h.SalesOrderID) AS LineSum
FROM Sales.SalesOrderHeader h;

-- Fast: join to a single aggregation
SELECT h.SalesOrderID, x.LineSum
FROM Sales.SalesOrderHeader h
JOIN (SELECT SalesOrderID, SUM(LineTotal) AS LineSum
      FROM Sales.SalesOrderDetail GROUP BY SalesOrderID) x
  ON x.SalesOrderID = h.SalesOrderID;
```

Result / Verify

The repeated inner execution collapses into one grouped scan/seek; runtime drops.

© PMSQLDBA

Scenario 70 DISTINCT masking a join that produces duplicates

ANTI-PATTERNS

Symptom

A query uses `SELECT DISTINCT` to "clean up" duplicate rows and is slow because of a large sort.

ROOT CAUSE

`DISTINCT` is often a band-aid over a join that fans out (one-to-many) and produces duplicates. The engine must materialize and de-duplicate a bloated set with an expensive Sort/Hash — the real problem is the join granularity.

FIX

Fix the join so it does not create duplicates (aggregate the many-side first, or use `EXISTS` when you only need existence), removing the need for `DISTINCT` and its sort.

```
-- Instead of DISTINCT after a fan-out join:
SELECT DISTINCT c.CustomerID
FROM Sales.Customer c JOIN Sales.SalesOrderHeader h ON h.CustomerID =
c.CustomerID;

-- Use EXISTS: no duplicates, no sort, seekable
SELECT c.CustomerID
FROM Sales.Customer c
WHERE EXISTS (SELECT 1 FROM Sales.SalesOrderHeader h WHERE h.CustomerID =
c.CustomerID);
```

Result / Verify

The distinct sort disappears; the semantics are clearer and the plan is cheaper.

> **NOTE — EXISTS vs IN vs JOIN+DISTINCT**
> For "does a related row exist", `EXISTS` short-circuits on the first match and avoids duplicates entirely — usually the best-performing and clearest choice.

© PMSQLDBA

Scenario 71 NOT IN with a NULL-able subquery returning nothing

ANTI-PATTERNS

Symptom

A `NOT IN (subquery)` unexpectedly returns zero rows or wrong results, and rewriting it changes both correctness and speed.

ROOT CAUSE

If the subquery can return `NULL`, `NOT IN` evaluates to `UNKNOWN` for every row (three-valued logic), so the whole predicate filters everything out. It is also harder for the optimizer to transform efficiently.

FIX

Use `NOT EXISTS`, which is NULL-safe and typically produces a better anti-semi-join plan.

```
-- Risky: NOT IN with possible NULLs
SELECT * FROM A WHERE A.k NOT IN (SELECT B.k FROM B);   -- wrong if B.k has NULLs

-- Correct and efficient
SELECT * FROM A WHERE NOT EXISTS (SELECT 1 FROM B WHERE B.k = A.k);
```

Result / Verify

Results become correct regardless of NULLs and the plan uses a clean anti-semi-join.

WARNING — A correctness bug, not just perf

This one silently returns wrong data. Standardize on `NOT EXISTS` for anti-joins so a stray NULL never empties your result set.

 © PMSQLDBA

Scenario 72 Paging with high OFFSET scanning everything

ANTI-PATTERNS

Symptom

Deep pagination (`OFFSET 100000 ROWS FETCH NEXT 50`) gets slower the further the user pages.

ROOT CAUSE

`OFFSET/FETCH` still reads and discards all rows up to the offset. Page 1 is cheap; page 2,000 reads 100,000 rows just to throw them away — cost grows linearly with the page number.

FIX

Use **keyset (seek) pagination**: remember the last key from the previous page and seek past it, so each page reads only its own rows regardless of depth.

```
-- Keyset pagination: pass the last seen key, seek forward
SELECT TOP (50) SalesOrderID, OrderDate, TotalDue
FROM   Sales.SalesOrderHeader
WHERE  SalesOrderID > @LastSeenId      -- seek past prior page
ORDER BY SalesOrderID;
```

Result / Verify

Every page costs the same small number of reads; deep pages are as fast as page one.

© PMSQLDBA

Scenario 73 Multi-statement table-valued function with a fixed cardinality estimate

ANTI-PATTERNS

Symptom

A query joining to a multi-statement TVF chooses a poor plan; the estimate for the function output is a fixed number regardless of actual rows.

ROOT CAUSE

A **multi-statement** TVF materializes into a table variable with no statistics, so the optimizer uses a fixed guess (1 row pre-2014, 100 rows 2014+). When the function actually returns many rows, the join type and grants are wrong.

FIX

Convert the function to an **inline** TVF (a single `RETURN SELECT`) so the optimizer can see through it and estimate properly; or on 2017+ rely on interleaved execution which uses the actual row count for MSTVFs.

```
-- Inline TVF: optimizer estimates like a view, not a black box
CREATE OR ALTER FUNCTION dbo.OrdersByCustomer(@c INT)
RETURNS TABLE AS
RETURN (
    SELECT SalesOrderID, OrderDate, TotalDue
    FROM   Sales.SalesOrderHeader
    WHERE  CustomerID = @c
);
```

Result / Verify

The function's estimate tracks reality; the surrounding plan corrects its join and memory decisions.

> **TIP — Inline beats multi-statement**
>
> Whenever a TVF can be expressed as one `SELECT`, make it inline. Multi-statement TVFs are optimizer black boxes and a classic cause of bad join plans.

© PMSQLDBA

Scenario 74 Nested views hiding an expensive base query

ANTI-PATTERNS

Symptom

A simple-looking `SELECT * FROM vReport` is slow; the view turns out to be built on other views built on other views.

ROOT CAUSE

View nesting hides complexity: the optimizer expands all layers into one large query, often materializing intermediate work, joining tables the outer query never needed, and defeating predicate pushdown. Each layer looked innocent; the expansion is enormous.

FIX

Flatten the query for the specific need — reference base tables directly with only the columns and joins required — or replace a heavy, frequently-read aggregate view with an indexed view (Part 11).

```
-- Instead of SELECT * FROM vReport (5 nested views),
-- write the targeted query against base tables with just the needed
joins/columns.
```

Result / Verify

The expanded plan shrinks: fewer joins, no needless materialization, and predicates reach the base tables.

© PMSQLDBA

Scenario 75 COUNT(column) vs COUNT(*) confusion and needless DISTINCT

ANTI-PATTERNS

Symptom

A count query is slower than expected or returns a surprising number because of `COUNT(col)` or an unnecessary `COUNT(DISTINCT ...)`.

ROOT CAUSE

`COUNT(col)` ignores NULLs (so it can differ from `COUNT(*)`), and `COUNT(DISTINCT col)` forces a distinct sort/hash. Using them without intent adds a sort or changes the result.

FIX

Use `COUNT(*)` for "how many rows" (it can use the narrowest index and skips the NULL check), and only use `COUNT(DISTINCT ...)` when you truly need distinct counting — backed by an index on that column.

```
SELECT COUNT(*) FROM Sales.SalesOrderHeader WHERE Status = 5;   -- fastest row
count
-- Distinct only when required, and index the column:
SELECT COUNT(DISTINCT CustomerID) FROM Sales.SalesOrderHeader;
```

Result / Verify

Row counts use a light index scan with no distinct sort; distinct counts are intentional and supported by an index.

© PMSQLDBA

Scenario 76 N+1 query pattern: aggregating in the application

ANTI-PATTERNS

Symptom

A screen issues one query for a list, then a separate query per row (hundreds of round-trips); total latency is dominated by network chatter and `ASYNC_NETWORK_IO`.

ROOT CAUSE

The application (often via an ORM) loads a parent list, then lazily fires one child query per parent — the "N+1" pattern. Each round-trip has fixed overhead, so hundreds of tiny queries cost far more than one set-based query.

FIX

Replace the per-row queries with a single set-based query that joins or pre-aggregates the children, returning everything in one round-trip. Configure the ORM to eager-load the related data.

```
-- One query returns parents with their aggregated children:
SELECT h.SalesOrderID, h.TotalDue, d.LineCount, d.LineTotalSum
FROM Sales.SalesOrderHeader h
JOIN (SELECT SalesOrderID, COUNT(*) LineCount, SUM(LineTotal) LineTotalSum
      FROM Sales.SalesOrderDetail GROUP BY SalesOrderID) d
  ON d.SalesOrderID = h.SalesOrderID;
```

Result / Verify

Hundreds of round-trips collapse to one; `ASYNC_NETWORK_IO` and total latency drop dramatically.

BEST PRACTICE — Let the database do set-based work

Round-trips are expensive. Push joins and aggregation into a single query rather than looping in application code — the database is built for set operations.

© PMSQLDBA

Part 9 — Parallelism

Parallelism helps big queries and hurts small ones. These scenarios tune when and how SQL Server spreads work across cores.

Scenario 77 CXPACKET waits from a large scan going parallel

PARALLELISM

Symptom

A big analytic query shows high `CXPACKET`/`CXCONSUMER` waits and consumes many cores, affecting other workloads.

ROOT CAUSE

The query's estimated cost exceeded the (often default, very low) **cost threshold for parallelism = 5**, so the optimizer parallelized a large scan. Parallel threads coordinating (and sometimes waiting on a skewed branch) generate `CXPACKET` waits.

Diagnose

Confirm the parallel operators and the server's cost threshold and MAXDOP.

```
SELECT name, value_in_use FROM sys.configurations
WHERE name IN ('cost threshold for parallelism','max degree of parallelism');
```

FIX

First reduce the work so the query no longer needs parallelism (index it down). At the server level, raise **cost threshold for parallelism** from the 1990s-era default of 5 to something like 25–50, and set **MAXDOP** appropriately for the core/NUMA layout instead of leaving it at 0.

```
EXEC sp_configure 'cost threshold for parallelism', 50; RECONFIGURE;
EXEC sp_configure 'max degree of parallelism', 8; RECONFIGURE;  -- per your
CPU/NUMA
```

Result / Verify

Trivial queries stop going parallel; genuinely large ones parallelize with less coordination overhead. `CXPACKET` falls to a reasonable level.

> **NOTE — CXPACKET is not automatically bad**
> Parallel data-warehouse queries naturally show `CXPACKET`. Treat it as a tuning signal only when paired with a low cost threshold, a bad estimate, or skew — not as something to eliminate outright.

 © PMSQLDBA

 © PMSQLDBA

Scenario 78 A scalar UDF silently serializing the whole plan

PARALLELISM

Symptom

A query that "should" parallelize stubbornly runs on one thread and is slow.

ROOT CAUSE

Certain constructs force a serial plan — most commonly a **scalar UDF** (pre-2019), but also inserting into a table variable and a few other operators. The presence of one serialization-forcing element makes the entire statement single-threaded.

FIX

Remove the serializing construct: inline the scalar UDF or convert it to an inline TVF (or use 2019 UDF inlining). Once the blocker is gone the optimizer can parallelize.

```
-- Replace the scalar UDF with inline logic or an inline TVF (see Anti-Patterns).
-- Confirm the plan's "NonParallelPlanReason" property is no longer set.
```

Result / Verify

The plan is allowed to go parallel; the `NonParallelPlanReason` property clears and runtime improves.

© PMSQLDBA

Scenario 79 Skewed parallel distribution (uneven Repartition Streams)

PARALLELISM

Symptom

A parallel query uses all threads but one thread does most of the work; runtime barely improves over serial.

ROOT CAUSE

Data skew on the distribution key means one thread receives most rows after a `Repartition Streams`, so the query is only as fast as its busiest thread. Parallelism is present but unbalanced.

Diagnose

In the actual plan, the parallel operator's per-thread row counts are wildly uneven (one thread ≫ the others).

FIX

Reduce the skew's impact: filter or pre-aggregate before the repartition, choose a more evenly-distributed key, or update statistics so the optimizer distributes better. For pathological skew, testing a different `MAXDOP` can help.

```
-- Pre-aggregate before the parallel exchange to even out the work
SELECT k, SUM(v) FROM (
    SELECT k, SUM(v) AS v FROM dbo.Fact GROUP BY k   -- local partial aggregate
) x GROUP BY k;
```

Result / Verify

Per-thread row counts even out; wall-clock time drops closer to the ideal parallel speedup.

© PMSQLDBA

Part 10 — Schema & Storage

Physical design decisions — key choice, data types, partitioning — set the ceiling for how fast queries can ever be. These scenarios raise that ceiling.

Scenario 80 Random GUID clustered key causing page splits and fragmentation

SCHEMA & STORAGE

Symptom

A table clustered on a `NEWID()` GUID fragments quickly, inserts are slow, and all nonclustered indexes are bloated.

ROOT CAUSE

A random GUID clustered key inserts rows into the **middle** of the B-Tree constantly, causing frequent page splits (fragmentation, extra log). Because the clustered key is copied into every nonclustered index as the row locator, a wide 16-byte GUID also bloats every nonclustered index.

FIX

Cluster on a narrow, ever-increasing surrogate (`INT/BIGINT IDENTITY`) and keep the GUID as a non-clustered unique column if the application needs it. If a GUID clustered key is unavoidable, use `NEWSEQUENTIALID()` to make inserts append-only.

```
-- Preferred: narrow ever-increasing clustered key
CREATE TABLE dbo.Thing (
    ThingId   BIGINT IDENTITY PRIMARY KEY CLUSTERED,   -- append-only
    PublicId  UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID() UNIQUE  -- keep GUID
nonclustered
);
-- If GUID must be clustered, at least make it sequential:
-- PublicId UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY CLUSTERED
```

Result / Verify

Fragmentation growth slows dramatically, inserts speed up, and nonclustered indexes shrink.

> **BEST PRACTICE — Narrow, unique, static, ever-increasing**
> These four properties define a good clustered key. GUIDs violate "narrow" and (with `NEWID()`) "ever-increasing", making them a poor default clustered key despite their convenience for distributed IDs.

© PMSQLDBA

 © PMSQLDBA

Scenario 81 Oversized data types wasting I/O and memory

SCHEMA & STORAGE

Symptom

Tables are far larger on disk than the data warrants; scans and backups are slow; memory grants are large.

ROOT CAUSE

Columns are defined much wider than needed — `NVARCHAR(4000)` for short codes, `BIGINT` for small enums, `CHAR(50)` padded fields. Wider rows mean fewer rows per 8-KB page, so every scan reads more pages and every grant is larger.

Diagnose

Compare declared sizes to actual max lengths and review rows-per-page.

```
SELECT name, system_type_name, max_length
FROM sys.dm_exec_describe_first_result_set(N'SELECT * FROM dbo.Wide', NULL, 0);
-- and MAX(LEN(col)) sampling to see true data widths
```

FIX

Right-size columns to the real data domain: `VARCHAR` instead of padded `CHAR`, `INT`/`TINYINT` for small ranges, `NVARCHAR` only when Unicode is required. Fewer bytes per row means more rows per page and less I/O everywhere.

```
-- Example right-sizing
ALTER TABLE dbo.Wide ALTER COLUMN StatusCode TINYINT NOT NULL;   -- was BIGINT
ALTER TABLE dbo.Wide ALTER COLUMN CountryISO CHAR(2) NOT NULL;   -- was
NVARCHAR(50)
```

Result / Verify

Table size, page reads, backup time, and memory grants all decrease.

© PMSQLDBA

Scenario 82 No partition elimination on a huge date-ranged table

SCHEMA & STORAGE

Symptom

Queries filtering by date on a very large partitioned table still scan all partitions.

ROOT CAUSE

Either the table is not partitioned on the filter column, or the predicate is written so partition elimination cannot apply (function on the partitioning column, or a parameter type mismatch). Without elimination, the engine touches every partition.

FIX

Partition on the date column used for filtering and keep predicates SARGable so the optimizer can eliminate non-matching partitions. Verify with the "Actual Partition Count" property.

```
-- Ensure the filter targets the partitioning column with a SARGable range
SELECT ... FROM dbo.FactSales
WHERE OrderDate >= '2024-01-01' AND OrderDate < '2024-02-01';
-- Plan property "Actual Partitions Accessed" should show only the January
partition(s).
```

Result / Verify

The plan accesses only the relevant partitions; reads scale with the date range, not the whole table.

> **NOTE — Partitioning is for manageability first**
>
> Partitioning helps sliding-window loads and maintenance more than raw query speed. For pure query performance, a good index often matters more — but elimination is a real bonus when the design fits.

© PMSQLDBA

Scenario 83 Heap with forwarding pointers inflating reads

SCHEMA & STORAGE

Symptom

A table with no clustered index gets slower to read over time; logical reads climb even though row counts are stable.

ROOT CAUSE

On a heap, when an UPDATE grows a row so it no longer fits its original slot, SQL Server moves the row and leaves a **forwarding pointer** behind. Every read that arrives via the old location must follow the pointer to the new page — an extra I/O per forwarded row, accumulating silently as the table is updated.

Diagnose

Check for forwarded records on the heap.

```
SELECT forwarded_record_count, avg_fragmentation_in_percent, page_count
FROM sys.dm_db_index_physical_stats
     (DB_ID(), OBJECT_ID('dbo.StagingHeap'), 0, NULL, 'DETAILED');
-- index_id 0 = heap; a high forwarded_record_count is the smoking gun
```

FIX

Add a clustered index (the durable fix — most transactional tables should have one). If the table must remain a heap, rebuild it to remove forwarding pointers.

```
-- Best: give the table a clustered index (ordered, no forwarding pointers)
CREATE CLUSTERED INDEX CIX_StagingHeap ON dbo.StagingHeap (Id);

-- If it must stay a heap, rebuild the heap to clear forwarded records
ALTER TABLE dbo.StagingHeap REBUILD;
```

Result / Verify

`forwarded_record_count` drops to zero and logical reads for the same queries fall.

> **NOTE — Heaps are for staging, not OLTP**
> Heaps suit write-once bulk-load landing tables. Any table that receives updates over time is better off clustered — it avoids forwarding pointers and gives seekable access.

© PMSQLDBA

Scenario 84 Over-normalization forcing many joins for every read

SCHEMA & STORAGE

Symptom

Common read queries must join six or more tables to assemble a single logical entity; plans are large and latency is high.

ROOT CAUSE

Excessive normalization scatters frequently-read attributes across many tables. Every read reassembles them with numerous joins, each adding cost and estimation risk. Normalization is correct for writes but can over-tax read-heavy paths.

FIX

For hot read paths, denormalize judiciously: a covering index that pulls the needed columns together, a materialized/indexed view, or a maintained summary table. Keep the normalized model authoritative; add the denormalized structure as an optimization.

```
-- Example: an indexed view materializing the common join/aggregate (see Part 11),
-- or a covering index that lets one table answer the hot query without extra
joins.
```

Result / Verify

The hot query touches far fewer structures; plan size and latency drop while the normalized model stays intact for writes.

> **NOTE — Normalize for correctness, denormalize for speed**
> Denormalization is a deliberate, measured optimization for specific read paths — not a license to abandon the normalized model. Maintain both and keep the normalized tables the source of truth.

© PMSQLDBA

Part 11 — Advanced

Columnstore, rowgroup health, and materialized aggregates address analytic workloads that rowstore indexes alone cannot.

Scenario 85 Rowstore aggregation that should be columnstore

ADVANCED

Symptom

A management report summing revenue across millions of detail rows takes seconds on a clustered-index scan.

ROOT CAUSE

Rowstore reads all columns of every row even when the aggregate needs only two, and processes in row mode. For large-scan aggregations this is inherently I/O- and CPU-heavy.

FIX

Add a **columnstore** index for the analytic path. It stores data by column with heavy compression and executes in **batch mode**, reading only the needed columns — typically an order of magnitude faster for aggregations. A nonclustered columnstore lets OLTP and OLAP coexist on one table.

```
-- Real-time analytics alongside OLTP: nonclustered columnstore
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_SOD
    ON Sales.SalesOrderDetail (ProductID, OrderQty, LineTotal);

SELECT ProductID, SUM(LineTotal) AS Revenue, COUNT(*) AS Lines
FROM   Sales.SalesOrderDetail
GROUP BY ProductID;   -- Columnstore Index Scan (batch mode) + Hash Aggregate
```

Result / Verify

The scan switches to a batch-mode columnstore scan; CPU and elapsed time drop sharply for the aggregation.

WARNING — Keep columnstore for OLAP

Columnstore is superb for scans/aggregations but poor for single-row OLTP lookups and frequent narrow updates. Keep the rowstore indexes for point queries and add columnstore only for the reporting path.

© PMSQLDBA

Scenario 86 Columnstore delta-store bloat from trickle inserts ADVANCED

Symptom

A columnstore table's analytic queries slow over time; many rows sit in uncompressed delta rowgroups.

ROOT CAUSE

Small "trickle" inserts (fewer than ~102,400 rows at a time) land in the **delta store** (rowstore) rather than compressed columnstore rowgroups. Too much delta-store data means queries read uncompressed rows and lose columnstore benefits; deletes also leave "deleted" rows behind until reorganized.

Diagnose

Inspect rowgroup states and sizes.

```
SELECT state_desc, COUNT(*) AS rowgroups, SUM(total_rows) AS rows
FROM sys.dm_db_column_store_row_group_physical_stats
WHERE object_id = OBJECT_ID('dbo.FactSales')
GROUP BY state_desc;
```

FIX

Load in large batches (≥ 102,400 rows) so data compresses directly into rowgroups, and periodically `REORGANIZE ... WITH (COMPRESS_ALL_ROW_GROUPS = ON)` to merge delta stores and purge deleted rows.

```
ALTER INDEX NCCI_SOD ON Sales.SalesOrderDetail
    REORGANIZE WITH (COMPRESS_ALL_ROW_GROUPS = ON);
```

Result / Verify

Delta-store rows shrink; a higher share of data is in `COMPRESSED` rowgroups and analytic scans speed up.

© PMSQLDBA

Scenario 87 Repeated identical aggregation better served by an indexed view

ADVANCED

Symptom

The same expensive aggregation (totals by product/day) runs constantly from many reports.

ROOT CAUSE

Each execution recomputes the aggregate from base rows. When the same grouped result is needed repeatedly and the base data changes far less often than it is read, recomputation is wasted work.

FIX

Materialize the aggregate with an **indexed view** (schema-bound view + unique clustered index). SQL Server maintains it automatically on writes and can even use it for queries that do not name the view (Enterprise "automatic matching").

```
CREATE VIEW dbo.vProductDaySales WITH SCHEMABINDING AS
SELECT sod.ProductID, CAST(soh.OrderDate AS date) AS OrderDay,
       COUNT_BIG(*) AS Lines, SUM(sod.LineTotal) AS Revenue
FROM   Sales.SalesOrderDetail sod
JOIN   Sales.SalesOrderHeader soh ON soh.SalesOrderID = sod.SalesOrderID
GROUP BY sod.ProductID, CAST(soh.OrderDate AS date);

CREATE UNIQUE CLUSTERED INDEX CIX_vPDS
    ON dbo.vProductDaySales (ProductID, OrderDay);   -- materializes the aggregate
```

Result / Verify

Reports read the pre-aggregated view; reads collapse. Weigh the added write cost of maintaining the view on every base-table change.

BEST PRACTICE — Indexed views suit read-heavy aggregates

They shine when the base tables are read far more than written and the aggregate is stable. On write-heavy tables the maintenance overhead can outweigh the read savings — measure both sides.

© PMSQLDBA

Scenario 88 Batch-mode execution not kicking in for an analytic rowstore query

ADVANCED

Symptom

A large aggregation over a rowstore table runs in row mode and is CPU-heavy, even on SQL Server 2019+.

ROOT CAUSE

Batch-mode processing (which handles ~900 rows per CPU instruction group and is far more efficient for aggregation) historically required a columnstore index. Without one, an analytic rowstore query stays in slower row mode.

FIX

On SQL Server 2019+ at compatibility level 150, **batch mode on rowstore** can apply automatically to qualifying analytic queries; ensure the compat level is set. Otherwise add a (nonclustered) columnstore index to enable batch mode explicitly.

```
ALTER DATABASE AdventureWorks2019 SET COMPATIBILITY_LEVEL = 150;   -- enables
batch mode on rowstore
-- Or enable it explicitly with a columnstore index for the analytic path:
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_SOD
    ON Sales.SalesOrderDetail (ProductID, OrderQty, LineTotal);
```

Result / Verify

The plan’s operators show “Actual Execution Mode = Batch”; CPU per aggregation drops substantially.

© PMSQLDBA

Scenario 89 Extreme write contention relieved by In-Memory OLTP ADVANCED

Symptom

A small, extremely hot table (session state, counters, queue) is a latch/lock bottleneck that indexing and isolation changes cannot fully resolve.

ROOT CAUSE

For the hottest tables, even optimal disk-based indexing hits latch and lock limits under massive concurrency. The disk-based storage engine's page latches become the ceiling.

FIX

Move the hot table to **memory-optimized** (In-Memory OLTP): lock-free, latch-free data structures with optimistic concurrency, optionally accessed via natively-compiled stored procedures. Scope it to the few tables that truly need it.

```
-- Requires a MEMORY_OPTIMIZED_DATA filegroup
CREATE TABLE dbo.SessionState (
    SessionId  UNIQUEIDENTIFIER NOT NULL PRIMARY KEY NONCLUSTERED,
    Payload    VARBINARY(8000) NOT NULL,
    UpdatedAt  DATETIME2 NOT NULL
) WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_AND_DATA);
```

Result / Verify

Latch/lock waits on the hot table vanish; throughput scales with concurrency far beyond the disk-based ceiling.

> **WARNING — Targeted, not wholesale**
>
> In-Memory OLTP has feature and memory constraints and is meant for a handful of extreme-contention tables — not the whole database. Size memory carefully and test durability/recovery on your VMSQL lab first.

© PMSQLDBA

Scenario 90 Distinct count over billions of rows: APPROX_COUNT_DISTINCT

ADVANCED

Symptom

A `COUNT(DISTINCT ...)` over an enormous table is slow and memory-hungry because of the exact distinct sort/hash.

ROOT CAUSE

Exact distinct counting must track every distinct value, needing large memory (and often a spill) on high-cardinality columns over huge tables. Exactness is expensive.

FIX

When a small error margin is acceptable (dashboards, trend metrics), use `APPROX_COUNT_DISTINCT` (2019+), which uses a HyperLogLog-style algorithm with fixed, small memory and ~2% error.

```
-- Exact (expensive):
SELECT COUNT(DISTINCT CustomerID) FROM Sales.SalesOrderHeader;

-- Approximate (fast, ~2% error, tiny fixed memory):
SELECT APPROX_COUNT_DISTINCT(CustomerID) FROM Sales.SalesOrderHeader;
```

Result / Verify

The approximate query runs in a fraction of the time and memory with an error well within reporting tolerance.

> **WARNING — Approximate means approximate**
>
> Never use `APPROX_COUNT_DISTINCT` where exact figures are required (billing, compliance counts). It is for large-scale analytics where a ~2% margin is acceptable.

© PMSQLDBA

Reference — Wait-Statistics Driven Tuning

Query-level tuning fixes individual statements; **wait statistics** tell you what the whole instance is waiting on, so you can decide *which* class of problem to chase first. Start any server-wide investigation here, then drill into the matching scenarios above.

```
-- Top waits since last restart (filtered of benign/idle waits)
SELECT TOP 20 wait_type,
       wait_time_ms / 1000.0 AS wait_s,
       signal_wait_time_ms / 1000.0 AS signal_s,
       waiting_tasks_count
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN (
   'CLR_SEMAPHORE','LAZYWRITER_SLEEP','RESOURCE_QUEUE','SLEEP_TASK',
   'SLEEP_SYSTEMTASK','SQLTRACE_BUFFER_FLUSH','WAITFOR','BROKER_TASK_STOP',
   'CHECKPOINT_QUEUE','REQUEST_FOR_DEADLOCK_SEARCH','XE_TIMER_EVENT',

'DIRTY_PAGE_POLL','HADR_FILESTREAM_IOMGR_IOCOMPLETION','SP_SERVER_DIAGNOSTICS_SLEEP')
ORDER BY wait_time_ms DESC;
```

Wait type	Usually means	Where to look
CXPACKET / CXCONSUMER	Parallelism coordination / skew	Part 9 — Parallelism
PAGEIOLATCH_*	Reading data pages from disk (I/O bound)	Parts 1–2 (indexing/SARGability), storage
RESOURCE_SEMAPHORE	Queries waiting for a memory grant	Part 6 — oversized grants
LCK_M_*	Blocking on locks	Part 7 — Concurrency
PAGELATCH_* on 2:x:x	tempdb allocation contention	Part 6 — tempdb files
SOS_SCHEDULER_YIELD	CPU pressure / long CPU-bound queries	Parts 1–8 (reduce CPU per query)
WRITELOG	Transaction-log flush latency	Storage / batch size (Part 7)
ASYNC_NETWORK_IO	Client not consuming results fast enough	App-side / SELECT * (Part 8)
HADR_SYNC_COMMIT	Sync-commit AG replica acknowledgement	AG topology / network (your VMSQL lab)

NOTE — Waits point, cards fix

Wait stats narrow the search; they do not fix anything by themselves. Use them to pick the right part of this runbook, then apply the scenario that matches the specific query or object.

© PMSQLDBA

Appendix — Diagnostic Toolkit

A. Symptom → likely cause quick index

Symptom you see	Most likely cause	Card / Part
Scan instead of seek	Missing index or non-SARGable predicate	Parts 1–2
Key/RID Lookup on fat arrow	Index not covering the query	Part 1
Implicit conversion warning	Data-type / collation mismatch	Part 2
Estimated <<>> Actual rows	Stale/missing stats, sniffing, variables	Part 3
Nested loop runs millions of times	Under-estimate on the outer input	Parts 3–4
Spill warning (Sort/Hash)	Grant under-estimate	Parts 4, 6
RESOURCE_SEMAPHORE waits	Over-sized memory grants	Part 6
Blocking chains	Pessimistic isolation / long tx / escalation	Part 7
Fast in SSMS, slow in app	SET-option / ARITHABORT mismatch	Parts 1, 5
Same query, wildly varying speed	Parameter sniffing	Parts 3, 5
Deep paging gets slower	OFFSET scanning discarded rows	Part 8
Single-threaded when it should scale	Scalar UDF / serial-forcing op	Parts 8–9

B. Essential diagnostic commands

```
-- Measurement
SET STATISTICS IO ON; SET STATISTICS TIME ON;   -- reads + CPU/elapsed per
statement
SET STATISTICS XML ON;                          -- actual plan as XML

-- Live activity & blocking
SELECT session_id, blocking_session_id, wait_type, wait_time, cpu_time,
logical_reads
FROM sys.dm_exec_requests WHERE session_id > 50;

-- Most expensive cached queries (by reads)
SELECT TOP 20 qs.total_logical_reads, qs.execution_count,
       qs.total_logical_reads/qs.execution_count AS avg_reads,
       SUBSTRING(st.text,(qs.statement_start_offset/2)+1,
         ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text)
           ELSE qs.statement_end_offset END - qs.statement_start_offset)/2)+1) AS
query
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
ORDER BY qs.total_logical_reads DESC;

-- Statistics health
SELECT s.name, sp.last_updated, sp.rows, sp.modification_counter
FROM sys.stats s CROSS APPLY sys.dm_db_stats_properties(s.object_id,s.stats_id) sp
```


© PMSQLDBA

```
WHERE s.object_id = OBJECT_ID('Sales.SalesOrderDetail');

-- Missing / unused indexes
SELECT * FROM sys.dm_db_missing_index_details;
SELECT * FROM sys.dm_db_index_usage_stats WHERE database_id = DB_ID();

-- Memory grants in flight
SELECT session_id, requested_memory_kb, granted_memory_kb, used_memory_kb
FROM sys.dm_exec_query_memory_grants;

-- Query Store: force a good plan
EXEC sp_query_store_force_plan @query_id = <id>, @plan_id = <id>;
```

C. The ten habits of durable tuning

1. Fix the **estimate** before the plan — most bad plans are bad statistics.
2. Keep predicates **SARGable**: no functions/arithmetic/conversions on indexed columns.
3. **Cover** hot queries with INCLUDE; do not over-index cold ones.
4. Match **data types** at every boundary (params, joins, FKs).
5. Prefer **set-based** logic over cursors and per-row functions.
6. Use **RCSI** for non-blocking reads instead of `NOLOCK`.
7. **Batch** large DML to avoid lock escalation and log bloat.
8. Raise **cost threshold for parallelism** off the default 5; set **MAXDOP** deliberately.
9. Turn on **Query Store**; force good plans and catch regressions.
10. **Measure** every change with STATISTICS IO/TIME — never tune on faith.

D. Glossary of tuning terms

Term	Meaning
SARGable	A predicate that can use an index seek (Search ARGument-able). Functions/conversions on the column break it.
Cardinality estimate	The optimizer's predicted row count for a step, derived from statistics.
Selectivity	How narrowly a predicate filters; high selectivity = few rows returned.
Covering index	An index containing every column a query needs, so no lookup to the base table is required.
Key Lookup / RID Lookup	An extra fetch to the clustered index (Key) or heap (RID) for columns not in the nonclustered index.
Residual predicate	A filter applied after reading rows, versus a Seek Predicate that navigates the B-Tree.
Parameter sniffing	Reusing a cached plan built for the first-seen parameter value, which may not suit later values.

© PMSQLDBA

Term	Meaning
Row goal	An optimizer assumption (from TOP/EXISTS) that it can stop after N rows, favouring early-out plans.
Spill	A Sort or Hash operation that exceeded its memory grant and wrote to tempdb.
Memory grant	RAM reserved for a query's Sort/Hash operators, sized from the row estimate.
Lock escalation	Automatic promotion of many row/page locks to a single table lock (~5,000-lock threshold).
RCSI	Read Committed Snapshot Isolation — versioned, non-blocking reads under the default isolation level.
Batch mode	A CPU-efficient execution mode (columnstore, or rowstore on 2019+) processing ~900 rows per step.
Query Store	Built-in history of queries, plans, and runtimes; enables regression detection and plan forcing.

End of runbook. *All examples target AdventureWorks2019 on SQL Server 2019+. Validate every change against an actual execution plan and STATISTICS IO/TIME, and rehearse on your VMSQL lab before promoting to a production or AG environment.*

 © PMSQLDBA